第 4 章　数字系统实验平台的使用

本章通过 4 个实验使读者熟悉本书使用的软、硬件实验开发平台和数字系统的开发流程。本章的重点首先是学习 ModelSim 软件、Vivado 软件和 FPGA 开发系统的使用方法，其次是学习仿真技巧和调试技术。最后，读者还可通过本章提供的实验代码体会数字系统的层次化设计风格。

因为，本教程使用了 ModelSim 和 Vivado 两个 EDA 开发软件，所以，应特别注意文件管理。具体做法：每个实验建 src、sim 和 vivado 三个子文件夹。src 文件夹存放源文件，即两个开发软件的公共文件。sim 和 vivado 两个子文件夹分别存放 ModelSim 工程和 Vivado 工程。

本教程使用两种 FPGA 开发板：Nexys Video Artix-7 FPGA 多媒体音视频智能互联开发系统（简称 Nexys Video 开发板）和 Basys3 口袋开发板。本章在介绍数字系统的开发流程时，以 Nexys Video 开发板为范例，只在某些步骤时指出使用 Basys3 口袋实验开发板的差别。因此，使用 Basys3 开发板读者留意两者微小差别。

实验 1　ModelSim 仿真软件的使用

一、概述

1．Xilinx 的 FPGA 的开发设计流程

Xilinx 的 FPGA 的开发设计流程，如图 4.1 所示，包括设计输入、功能仿真、综合、实现、时序仿真、下载配置和在线调试等芯片编程与调试等主要步骤。

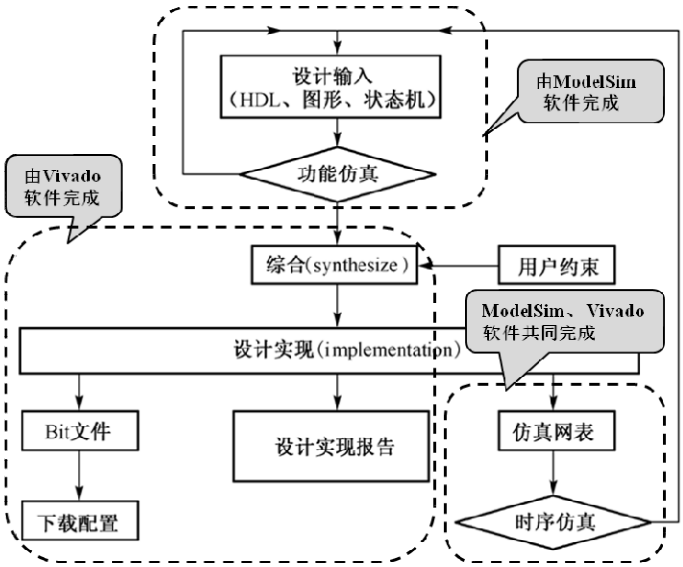


图 4.1　Xilinx 的 FPGA 开发流程图

1）设计输入（design entry）

Xilinx 的 FPGA 的开发软件 Vivado 采用基于工程的分层次管理，支持硬件描述性语言（HDL）、逻辑图和状态图等混合输入方式。本教程只采用 Verilog HDL 硬件描述性语言进行设计输入，因此设计可采用 ModelSim 或 Vivado 软件的编辑器，也可采用 Notepad++、UltraEdit 等专业代码编辑软件，建议使用后者。

2）功能仿真（behavioral simulation）

功能仿真的主要目的是验证设计文件的逻辑功能是否正确，即在理想状态下（不考虑延迟），验证电路的功能是否符合设计的要求。

虽然 Xilinx 开发软件 Vivado 自带仿真器，且功能也不错，但 Vivado 对电脑硬件要求较高，且运行速度慢。另外，功能仿真这一过程会经常修改设计代码，对仿真速度、仿真手段要求极高。而 ModelTech 公司的 ModelSim 软件是专业仿真软件，其在仿真速度、仿真手段更具优势，且对电脑硬件要求较低，所以本教程采用 ModelSim 软件进行仿真。

3）用户约束（user constraints）

在进行高速数字电路设计时，设计者经常需要对时序、I/O 引脚位置和区域等附加约束，以便控制综合、实现过程，使设计满足运行速度、面积和引脚位置等方面的要求。这些约束条件存放在用户约束文件（.XDC 文件）中，用户可随时编辑修改。

4）综合（synthesize）

综合是使用 HDL 设计的重要环节，是将 HDL 描述转换为用基本门电路、RTL 和厂家提供的基本单元进行描述的过程。

5）设计实现（implementation）

Vivado 的实现功能包括了逻辑优化(opt_design)、映射（map）、布局（place_design）和布线（route_design）等过程。

6）时序仿真（timing simulation）

时序仿真是在将设计适配到 FPGA 芯片后的仿真验证方式。时序仿真在严格的仿真时间模型下，模拟芯片的实际运作。仿真时间模型将最基本的门级时延计算在内，从而可有效地分析出设计中的竞争和冒险。在多数情况下，经过时序仿真验证后的设计基本上与实际电路是一致的。

7）下载配置（programming）

下载功能包括了 BitGen，用于将布局布线后的设计文件转换为比特流（bitstream）文件。还包括了 iMPACT 功能，用于进行设备配置和通信，控制程序烧写到 FPGA 芯片中。

本教程将用两个实验介绍 FPGA 开发设计流程，在实验 1 介绍功能仿真这一步骤，实验 2 介绍综合、实现、下载配置等开发过程。

2. ModelSim 软件简介

ModelSim 是一种简单易用、功能强大的逻辑仿真工具，具有广泛的应用。仿真分为功能仿真、门级仿真、时序仿真。

功能仿真也称为行为仿真或前仿真，即在完成一个设计的代码编写工作之后，可以直接对代码进行仿真，检测源代码是否符合功能要求。这时，仿真的对象为 HDL 代码，比较直观，速度比较快，可以进行与软件相类似的多种手段的调试。在设计的最初阶段发现问题，可以节省大量的精力。

功能仿真需要以下文件。

（1）HDL 源代码：可以是 VHDL 语言或 Verilog HDL 语言。

（2）测试激励代码（testbench）：根据要求设计输入、输出的激励程序，由于不需要进行综合，书写具有很大的灵活性。

（3）仿真模型/库：根据设计内容调用的器件供应商提供的模块，如 DCM、FIFO、ROM、RAM 等。

本实验对 ModelSim 的功能仿真方法进行入门性的介绍，在实验 3 介绍具有 IP 内核的数字系统的 ModelSim 仿真，并在后续的实验根据需要陆续介绍 ModelSim 仿真的一些使用技巧。

二、实验设备

装有 Vivado 软件和 ModelSim SE 软件的计算机。

三、设计代码

1．实验任务

设计并实现一个流水灯电路，要求：

（1）8 个发光二极管间隔 0.5s 轮流点亮。

（2）设置一个 direction 开关控制流水灯流水灯点亮方向。direction=0 时，向右轮流点亮；direction=1 时，向左轮流点亮。

（3）设置一个 reset 按钮，复位时，最右边灯亮。

2．电路总体框图

根据设计要求，流水灯电路可由三个子模块组成，如图 4.2 所示。分频器模块产生周期为 0.5 秒的脉冲信号 pluse2Hz，该信号决定流水灯的流动速度。计数器模块为三位可逆计数器，该模块决定流水灯的流动方向，direction=0，减法计数（向右轮流点亮）；direction=1，加法计数（向左轮流点亮）；译码器模块则根据计数状态 q 点亮相应编号的发光二极管。

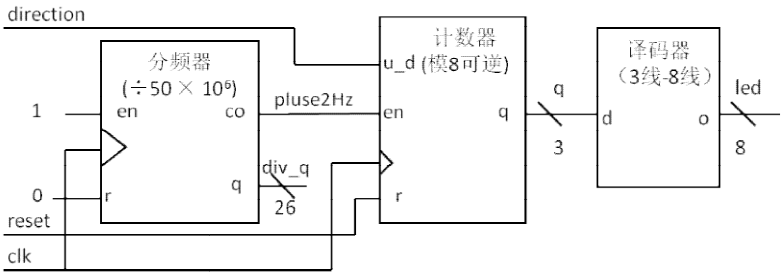


图 4.2　流水灯的顶层电路连接图

3．Verilog HDL 代码

由于电路具有大分频比的分频器模块，实时仿真基本不可能也没必要。因此，一般做法，仿真时将分频器改为小分频比的分频器模块，以减少仿真时间。具体解决方案，在顶层模块设置参数 sim，sim 默认值为 0。sim 取值决定分频器的分频比：sim=0，分频比设置为 50×10^6；而 sim=1，分频比设置 10。在测试代码，调用 top 模块，传递 sim 值为 1，这样就减少了仿真时间。另外，顶层模块中的 sim 默认 0 值不影响电路综合和实现。

下面，给出实验的部分 Verilog HDL 代码，除了理解电路原理外，读者还应重点理解参数 sim 的作用。

（1）流水灯电路的顶层模块

```
module FlowingWaterLight(clk,reset,direction,led);
   parameter   sim=0;// 默认综合方式
   input clk,reset,direction;
   output[7:0] led;
////////////////////////////////////////////////////////////////////////////////////////////////////
//n 进制计数器/分频器实例，sim=0 时，分频比 n=5000_0000；计数器位数
//counter_bits=26;而 sim=1 时，分频比 n=10；计数器位数 counter_bits=4;
////////////////////////////////////////////////////////////////////////////////////////////////////
    wire pulse2Hz;
    counter_n #(.n(sim?10:5000_0000),.counter_bits(sim?4:26))
         div_inst(.clk(clk), .en(1), .r(0), .q(),.co(pulse2Hz));
  //-------------------     可逆计数器实例    ------------------//
     wire[2:0] q;
     counter_up_dn #(.counter_bits(3) )   counter_up_dn_inst(
       .clk(clk),.en(pulse2Hz),.u_d(direction),.r(reset),.q(q));
  // -------------------     3 线-8 线器实例    ------------------//
     decode   decode_inst(.din(q),.out(led));
endmodule
```

（2）n 进制计数器/分频器模块

```
  module counter_n(clk,en,r,q,co);
  parameter   n=2; //参数 n 表示计数的模
  parameter   counter_bits=1;//参数 counter_bits 表示计数的位数
  input     clk,en,r ;
  output    co;
  output     [counter_bits-1:0]    q;
  reg [counter_bits-1:0]    q=0;
  assign    co=(q==(n-1)) && en;//进位
  always @(posedge clk)
    begin
      if(r) q=0;
```

```
        else if(en)   //en=1,计数    en=0,保持
                begin if(q==(n-1))   q=0 ; else   q=q+1; end
      end
   endmodule
```

（3）测试代码（testbench）

```
 `timescale 1ns/100ps
module FlowingWaterLight_tb;
parameter DELY=10;
reg    clk, reset, direction;
wire [7:0]        led;
//顶层模块实例，传递 sim 参数 1，改变分频比，减少仿真时间
FlowingWaterLight   #(.sim(1))   FlowingWateLight_inst (
      .clk(clk),.reset(reset),.direction(direction),.led (led));
//产生时钟波形
initial begin clk = 0; forever #5 clk = ~clk;end
///产生 reset、 direction 波形
initial     begin
     reset = 0; direction = 1;
     #(DELY/2+1)reset = 1;
     #(DELY)reset = 0;
     #(DELY*80) direction = 0;
     #(DELY*90)$stop;   end
endmodule
```

四、实验步骤

ModelSim 运行方式有以下 4 种。

（1）用户图形界面模式。

（2）交互式命令行模式，不显示 ModelSim 的可视化界面，仅通过命令控制台输入的命令完成所有工作。

（3）Tcl 和宏模式，编写可执行文件（扩展名为 do）或者 Tcl 语法文件。

（4）批处理模式。所有操作都在后台进行，用户看不到 ModelSim 的界面，也不需要交互式输入命令。当工程很大，文件比较多时，用批处理比较方便。直接运行批处理文件，在后台调用 ModelSim，执行 ModelSim 的脚本文件*.do，完成操作。

由于本实验为入门性介绍，且受篇幅限制，因此只介绍用户图形界面模式的仿真步骤。

1．下载文件

（1）从网站中下载 lab1_2_FlowingWaterLight 文件夹至硬盘里，存放在 e:\SystemDesignLab 文件下。lab1_2_FlowingWaterLight 共三个子文件夹，其中 src、sim 和 vivado 三个文件夹分别存放源代码、ModelSim 工程和 Vivado 工程。

src 有顶层设计代码 FlowingWaterLight.v、任意进制计数器/分频器设计代码 conter_ n.v、

任意位数可逆计数器设计代码 conter_up_down.v、译码器模块设计代码 decode.v 和顶层测试代码 FlowingWaterLight_tb.v。为了更好地学习 ModelSim 使用方法，在这些文件中，作者人为地设置了三处错误。

2．启动 ModelSim

启动后进入 ModelSim 的主窗口，主要由工作区、命令窗口、信息显示区、菜单栏和工具栏等几部分组成，如图 4.3 所示。

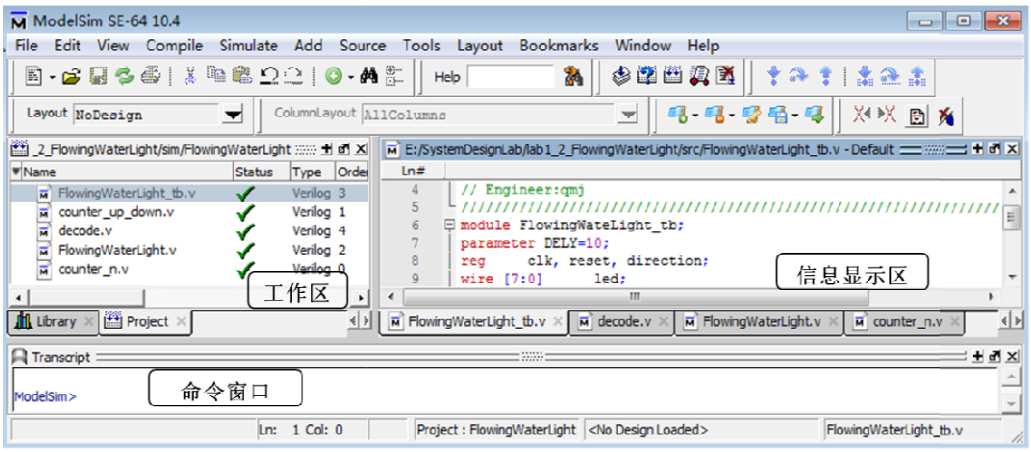


图 4.3　ModelSim 的主窗口

3．建立工程

在 ModelSim 的主窗口中，选择菜单 File⇨New⇨Project 命令；弹出如图 4.4 所示的 Create Project 对话框，在 Project Name 文本框中填写工程名：FlowingWaterLight。Project Location 文本框是工程文件夹路径，可通过 Browse 按钮来选择或改变文件夹路径，本例文件夹路径为 E:/SystemDesignLab/lab1_2_FlowingWaterLight/sim。

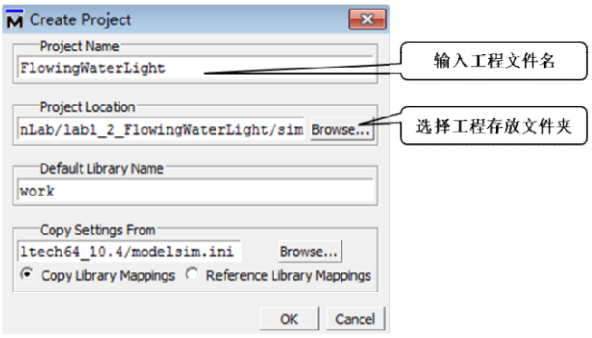


图 4.4　建立工程的对话框

注意：工程名和存储的路径中不能出现中文和空格，建议工程名称以字母、数字、下划线来组成。

4．给工程加入文件

单击 Create Project 对话框中的 OK 按钮后，ModelSim 自动弹出 Add items to the Project 对话框，如图 4.5 所示。选择 Add Exsiting File 后，弹出图 4.6 所示的 Add file to Project 对话框，根据相应提示将文件加到该 Project 中。本例需要加入 FlowingWaterLight.v、~~dffr.v、~~counter_n.v counter_up_down.v、decode.v 和 FlowingWaterLight_tb.v 等 5 个文件。这些文件均存在文件夹 E:\SystemDesignLab\lab1_2_FlowingWaterLight\src 下。注意，每次可加入多个文件。

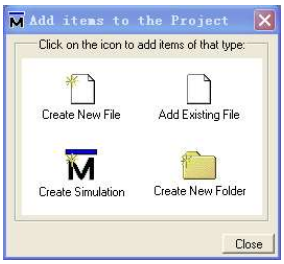


图 4.5　Add Items to the Project 窗口

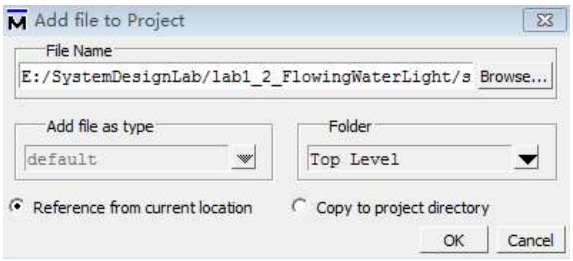


图 4.6　添加文件对话框

添加文件后的主界面如图 4.7 所示，工作区中文件的状态“？”表示未编译或修改后未编译；“√”表示编译通过；而“×”表示有错，编译未通过。

在工作区中，双击文件名，可在信息区查看相应代码。

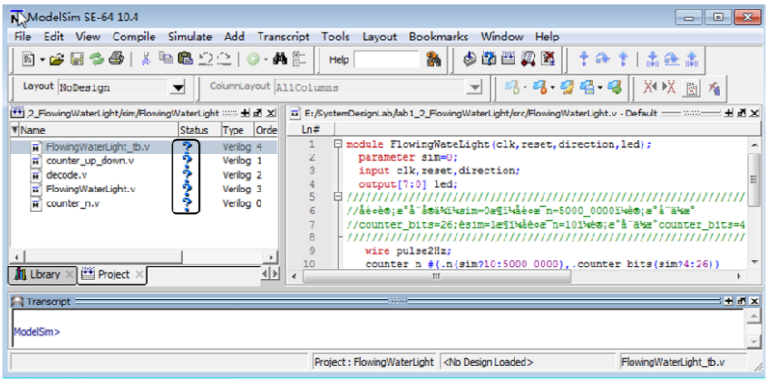


图 4.7　添加文件后的主界面

5．编译

执行菜单中 Compile⇨Compile All 命令来编译所有文件。编译后，在命令窗口会显示编译结果，同时各文件的 Status 栏中也会标记编译结果，如图 4.8 所示。

小提示：如文件的 Status 栏未标记编译结果，可在工作区右击，在弹出的快捷菜单有选择 Update 命令更新显示。

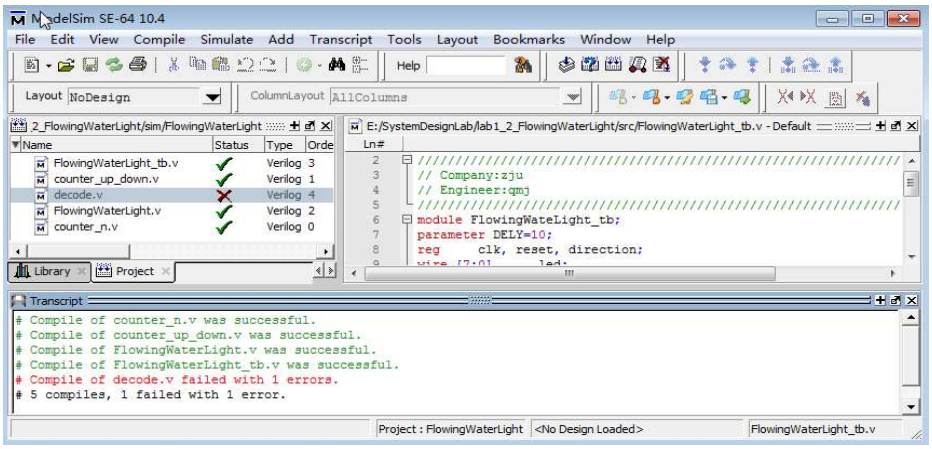


图 4.8　编译结果

图 4.8 所示可看出，模块 decode.v 编译出错。查找错误方法如图 4.9 所示，共有下列几个步骤。

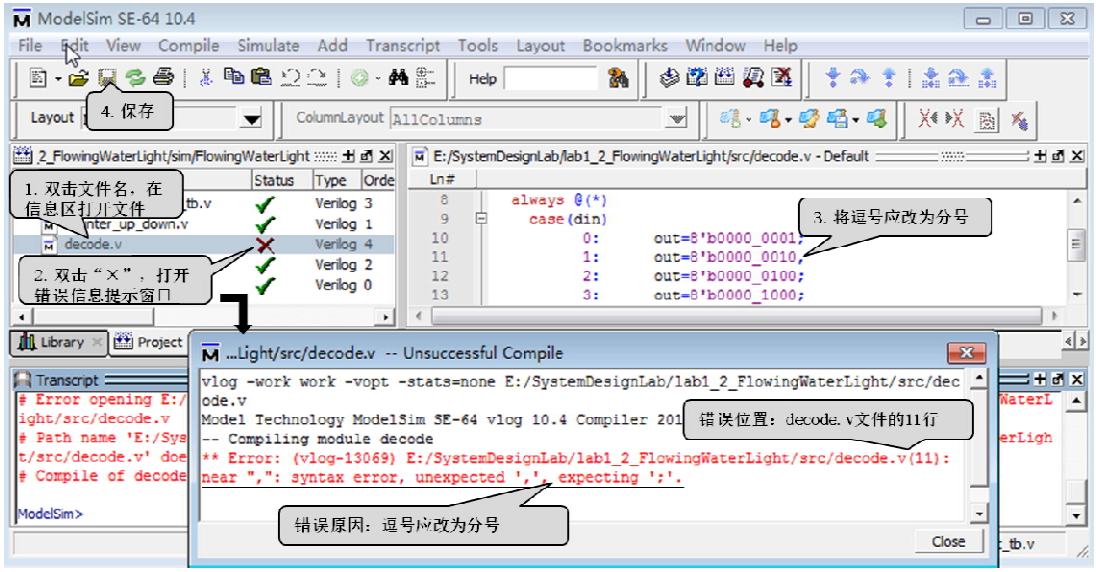


图 4.9　查错过程

（1）在工作区中，双击文件，在信息区打开文件。

（2）在文件的 Status 栏中双击“×”，打开错误信息提示窗口。

（3）根据错误提示的位置和原因，修改代码错误。本例，应将 decode.v 的第 11 行末尾逗号改为分号。

（4）保存文件，保存后，Status 栏中 “×”将变为“？”。

（5）修改文件后，应重新编译。在工作区选单击选中 decode.v，执行菜单中 Compile⇨Compile Selected 命令。

当所有文件有 Status 栏中都显示 “√”时，编译才算完成。

6．装载文件

如图 4.10 所示，单击工作区中 Library 标签，展开 work 库，work 库为当前的工作库。右击测试文件 FlowingWaterLight_tb，在快捷菜单中选择 Simulate without Optimization 命令 。

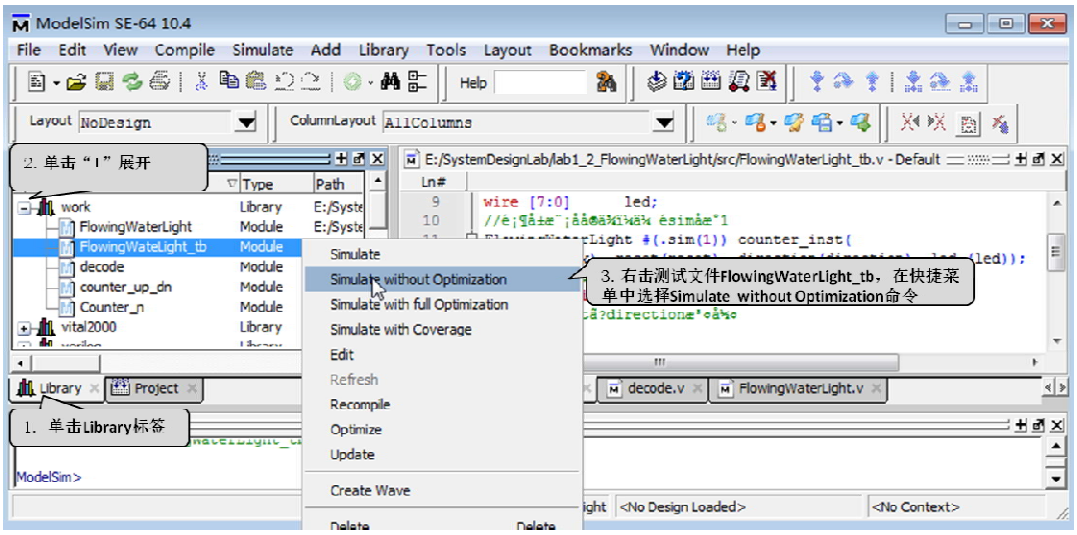


图 4.10　装载文件

因为代码中存在错误，所以装载失败。装载失败原因一般有两类，一是调用子模块时，模块名、参数名及端口名称错误等，二为变量类型定义错误。命令区会显示错误信息，如图 4.11 所示。本例错在调用子模块时，模块名出错，任意分频比计数器的模块名为 counter_n，而顶层模块调用时错用了 Counter_n。这里说明一下，Verilog HDL 以分号区隔语句，所以第 10~11 行为一条语句。这一条语句中的任何错误，大多情况下，提示错误位置放在最后一行。

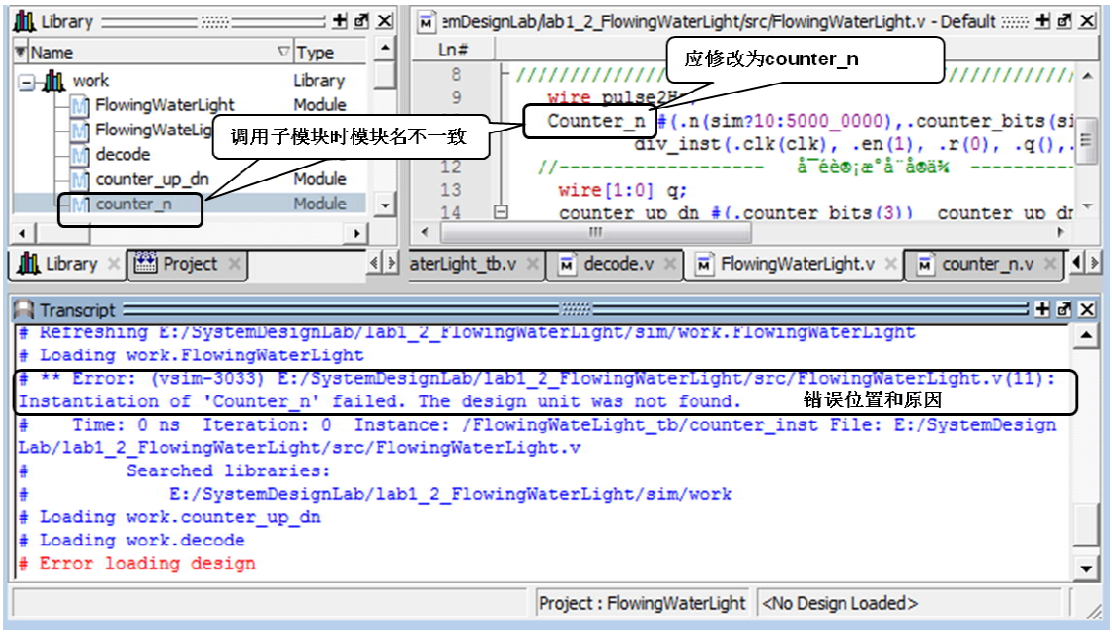


图 4.11　装载错误信息

将 FlowingWaterLight.v 文件的第 10 行的 Counter_n 改为 counter_n，保存文件，重新编译。

然后重新装载测试文件 FlowingWaterLight_tb，当工作区出现 sim 选项卡时，如图 4.12 所示，表示仿真模块装载成功。同时出现 Object 窗口，该窗口列出 sim 选项卡中选中模块的信号，包括端口信号、中间变量和参数等。

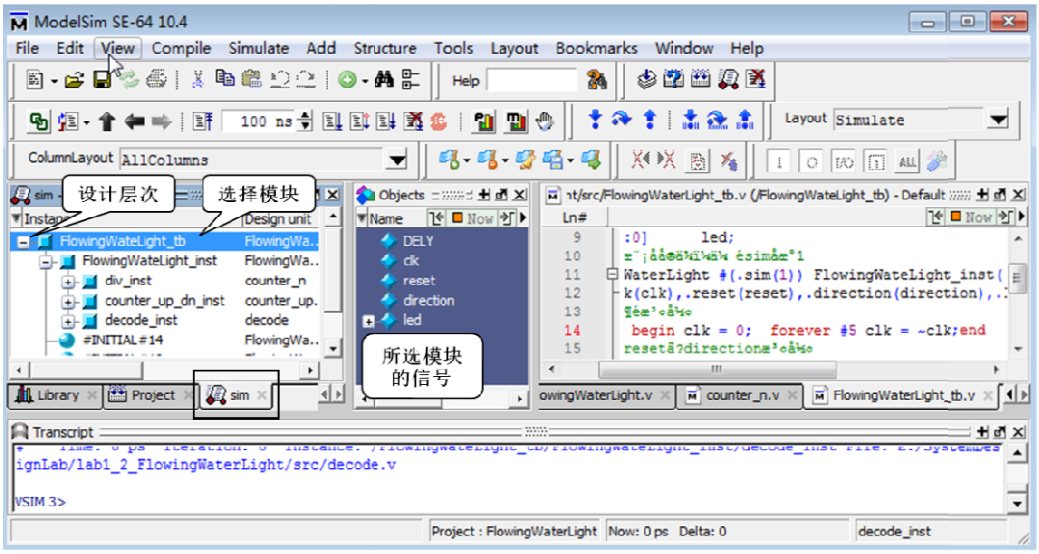


图 4.12　装载成功

7．仿真设置

如图 4.13 所示，在工作区的 sim 选项卡中 ，选中测试模块 FlowingWaterLight_tb，则在 Objects 窗口显示选中的 FlowingWaterLight_tb 模块中的信号。可右击 Objects 窗口，通过快捷菜单 Add to ⇨Wave⇨Signals in Region 命令将 Object 窗口所有信号加入 Wave 窗口，信息区将会出现波形窗口，如图 4.14 所示。

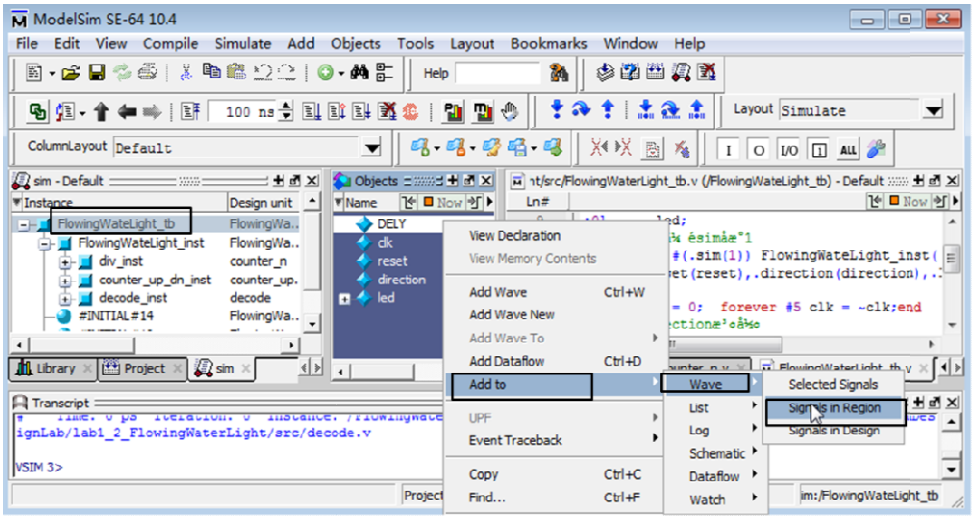


图 4. 13　加入测试信号至波形窗口

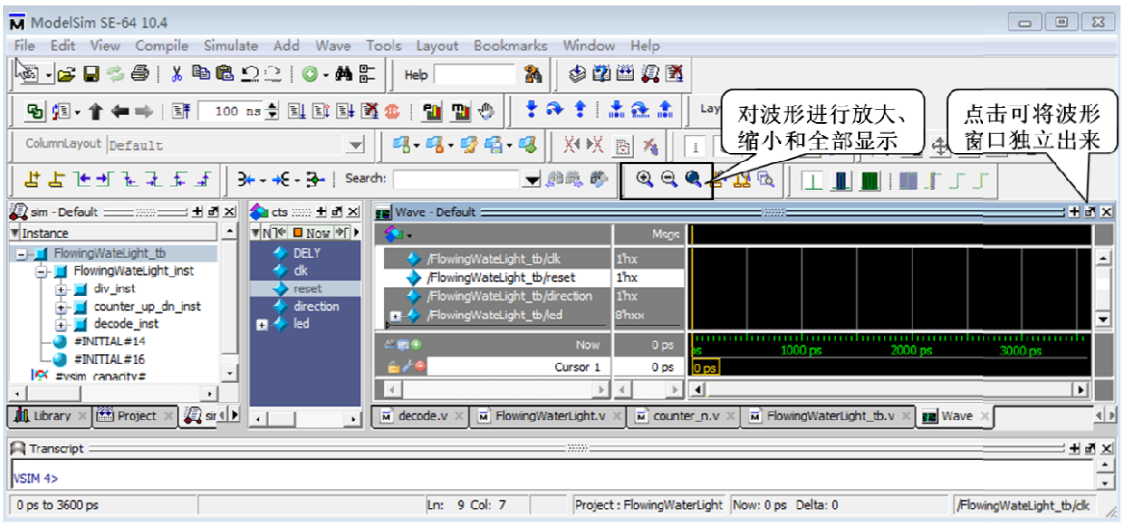


图 4. 14　加入测试信号后的波形窗口

8．仿真

1）设置信号的编码方式

ModelSim10.4 对信号默认显示方式为 16 进制，为了增加仿真结果的可读性，本例中 led 信号用二进制编码方式较为合适。在波形窗口右击 led 信号，在弹出的快捷菜单执行 Radix⇨Binary 命令，将 led 信号编码方式改为二进制。

2）执行仿真

选择菜单 Simulate⇨Run⇨Run-ALL 命令，得到图 4.15 所示的仿真波形（由于篇幅原因，只显示部分波形），从仿真波形可看出，led 高 4 位始终为 0，设计存在错误。下面将通过调试手段，查找设计错误。

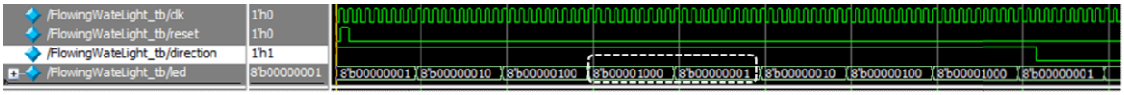


图 4.15　仿真波形之一

首先，观察顶层设计的中间变量 pulse2Hz 和 q 的波形，以确定设计错误发生在哪个模块，方法如下。

①如图 4.16 所示，在工作区的 sim 选项卡中，选中顶层模块 FlowingWaterLight，可在 Objects 窗口内选定要观察的信号 pulse2Hz 和 q，通过右键快捷菜单的 Add Wave 命令加到波形 Wave 窗口。也可直接将 Objects 窗口中的信号“拖至”波形 Wave 窗口。

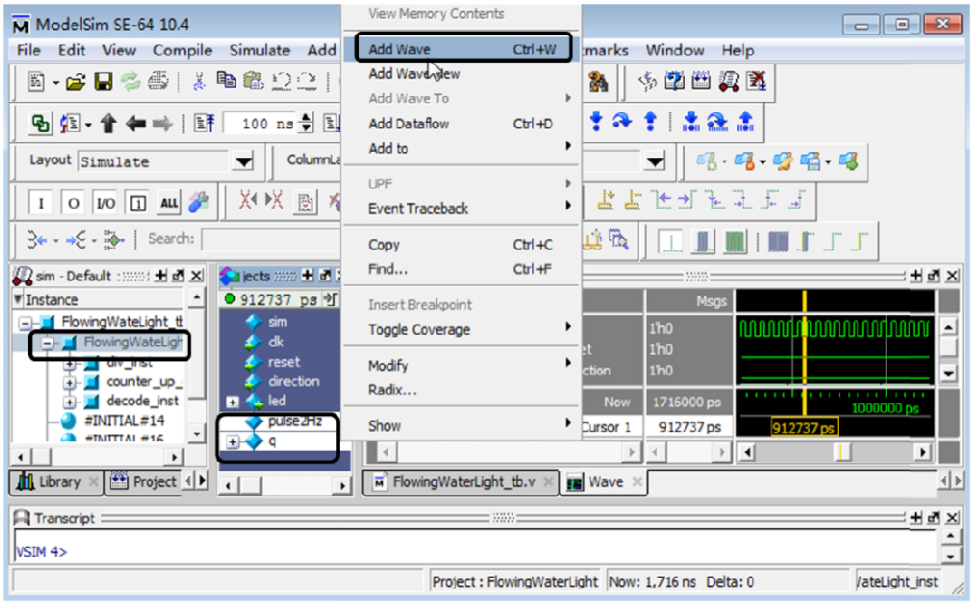


图 4. 16　加入顶层设计的信号方法

②执行菜单 Simulate ⇨Restart 命令，在弹出的 Restart 窗口中点击 OK 按钮。

③执行菜单 Simulate⇨Run⇨Run-ALL 命令，再次仿真。

仿真波形如图 4.17 所示，从仿真波形可知，pulse2Hz 和 q 波形均正确，基本可确定设计错误发生在 decode 模块。

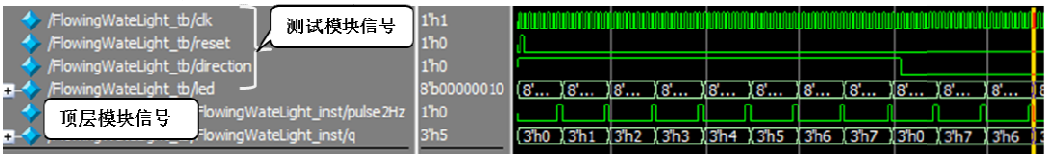


图 4.17　仿真波形之二

其次，根据上面分析，加入 decode 模块的全部信号，重新仿真，得到如图 4.18 的仿真波形。仔细分析波形，可看出 decode 模块的 din 信号出错（din 信号应与顶层的 q 信号一致），主要是 din 信号位宽不对。

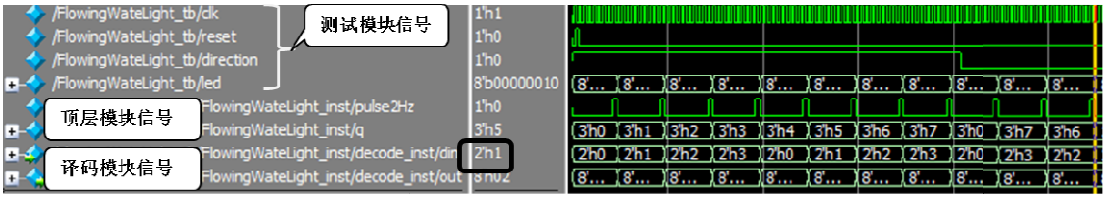


图 4.18　仿真波形之三

最后，打开 decode.v 文件，将第 6 行语句“input[1:0]　din ;”修改为“input[2:0]　din ;”。**保存修改后的 decode.v 文件；重新编译 decode.v 文件；执行菜单 Simulate ⇨Restart 命令复位；执行菜单 Simulate⇨Run⇨Run-ALL 命令，重新仿真。**得到如图 4.19 的仿真波形。

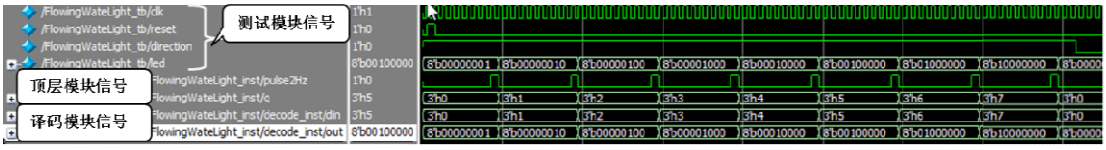


图 4.19　最后的仿真结果

分析图 4.19 所示的仿真波形，仿真结果符合设计要求，至此，功能仿真结束。关闭 ModelSim 软件，我们将在实验 2 继续完成流水灯实验。

9．仿真注意事项

①设计代码修改后，一定要先保存，再重新进行编译。

②重新仿真前，必须先执行菜单 Simulate ⇨Restart 命令进行复位，再执行菜单 Simulate⇨Run⇨Run-ALL 命令进行重新仿真。

最后说明一下，ModelSim 软件功能非常强大，上面介绍只是入门知识。读者在此基础上，多实践操作，以便更灵活地使用仿真手段。

五、思考题

（1）什么是仿真？常用的仿真软件有哪些？

（2）功能仿真与时序仿真有什么区别？两者的目的是什么？

（3）功能仿真需要哪些文件？

（4）ModelSim 中的各个窗口的作用是什么？除了波形窗口外，其他窗口对程序的调试有什么帮助？

（5）如何在 ModelSim 中查看被仿真文件的非端口信号?

实验 2　Vivado 软件的使用

一、Vivado 软件简介

Vivado 软件是 FPGA 厂商 Xilinx 公司的集开发工具套件，包括高度集成的设计环境和新一代从系统到IC级的开发工具，这些均建立在共享的可扩展数据模型和通用调试环境基础上。这也是一个基于 AMBA AXI4 互联规范、IP-XACT IP 封装元数据、工具命令语言(TCL)、Synopsys 系统约束(SDC) 以及其它有助于根据客户需求量身定制设计流程并符合业界标准的开放式环境。Xilinx 构建的的 Vivado 工具把各类可编程技术结合在一起，能够扩展多达 1 亿个等效 ASIC 门的设计。Vivado 软件主要特点有。

1.完整的设计工具。

设计全部流程完成在一个工具中，综合时就可考虑整个设计网表，在设计的各个阶段都需要检查各种约束文件，设计的整体性得到保证。

2.更优的 PC 资源使用方式。

开发软件是多个工具组合在一起，各工具交接时，产生临时文件来保存上一步的执行结果。Vivado 软件的设计思想是，直接保存在内存中，这避免了反复读写硬盘的麻烦。这一效果，对于越大的设计，体现得更明显。

3.更好的时序收敛。

Vivado 软件在时序设计、布线延时、资源等多方面时行综合考虑，具有更好的时序收敛。

4. Vivado 软件全面支持 Tcl 脚本。

5.支持 System Verilog。

二、实验内容

在实验 1 的基础上，继续介绍流水灯实验的综合、实现和下载配置过程。

三、实验设备

（1）装有 Vivado、ModelSim SE 软件的计算机。

（2）Nexys Video 开发板或 Basys 3 开发板。

四、实验步骤

Vivado 设计分为 Project Mode 和 Non-project Mode 两种模式，一般简单设计中，常用的是 Project Mode。在本实验中，我们将以一个简单的实验案例，一步一步地完成 Vivado 的整个设计流程。

1．建立 Vivado 工程文件

（1）启动 Xilinx Vivado 系统软件，可通过桌面 Vivado 2017.1 快捷方式或开始菜单中 Xilinx Design Tools Vivado 2017.1 下的 Vivado 2017.1 打开软件，开启后，软件如图 4.20 所示。

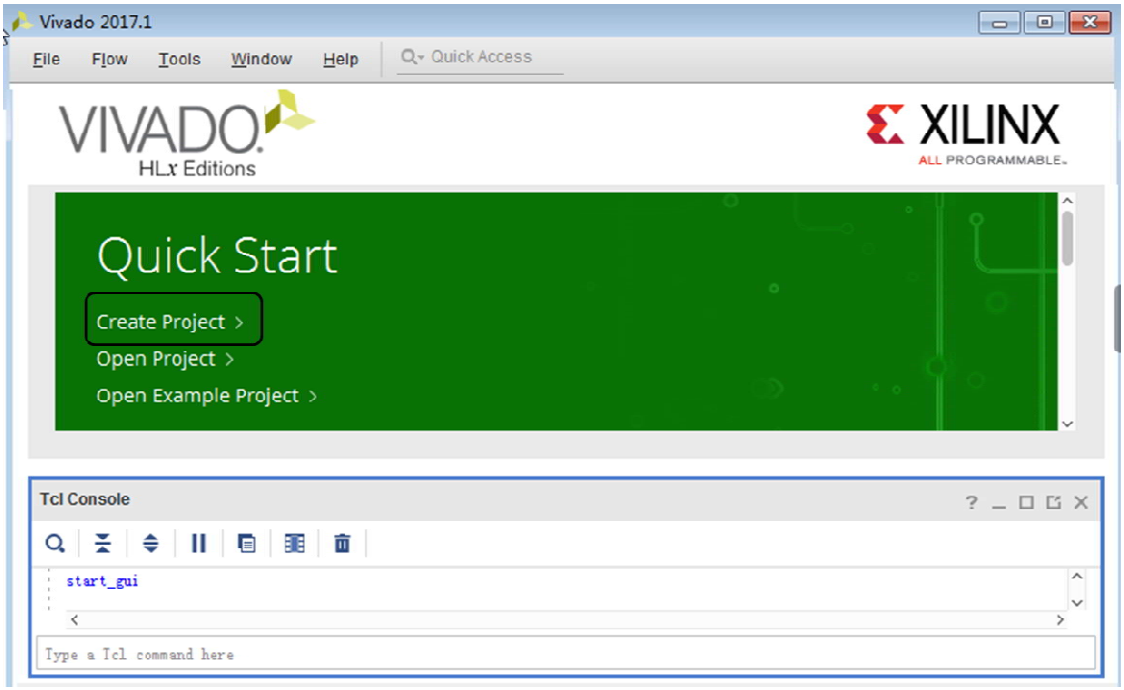


图 4.20　Vivado 起动界面

（2）单击上述界面中 Create Project 图标，弹出新建工程向导，如图 4.21 所示，点击 Next，进入填写工程名界面，如图 4.22 所示。

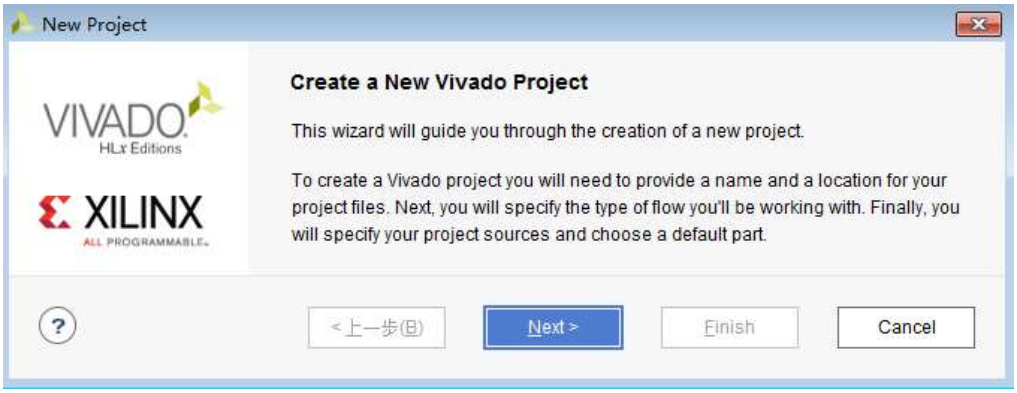


图 4.21　Vivado 新建工程向导之一

（3）在图 4.22 中，输入 Vivado 工程名称：FlowingWaterLight；选择工程存储路径：E:/SystemDesignLab/lab1_2_FlowingWaterLight/vivado；不要勾选 Create project subdirectory 选项。设置完成后，点击 Next，进入工程类型选择界面。

注意：工程名称和存储路径中不能出现中文和空格，建议工程名称以字母、数字、下划线来组成。



图 4.22　Vivado 新建工程向导之二

（4）在图 4.23 中，选择 RTL Project 一项，并勾选 Do not specify sources at this time，勾选该选项目的是为了跳过在新建工程的过程中添加设计源文件。点击 Next，进入 FPGA 目标器件选择界面。



图 4.23　Vivado 新建工程向导之三

（5）FPGA 目标器件与使用的 FPGA 开发平台有关，本教程将用到两种型号的 FPGA 开发板： Nexys Video 和 Basys 3；FPGA 目标器件分别为 Artix-7 xc7a200tfbg484-1 和 Artix-7 xc7a35tcpg236-1。教程将以 Nexys Video 开发板为主介绍开发过程。

图为 4.24 为 Nexys Video 开发板的 FPGA 目标器件选择的示意图，依次选 Family 为 Artix-7，封装形式（Package）为 fbg484，速度等级（Speed grade）为-1。并在 part 列表中选择 xc7a200tfbg484-1，点击 Next，进入信息确认界面。

注意：若使用 Basys 3 开发板，则需选择 Artix-7 xc7a35tcpg236-1 器件：Family 为 Artix-7，封装形式（Package）为 cpg236，速度等级（Speed grade）为-1。

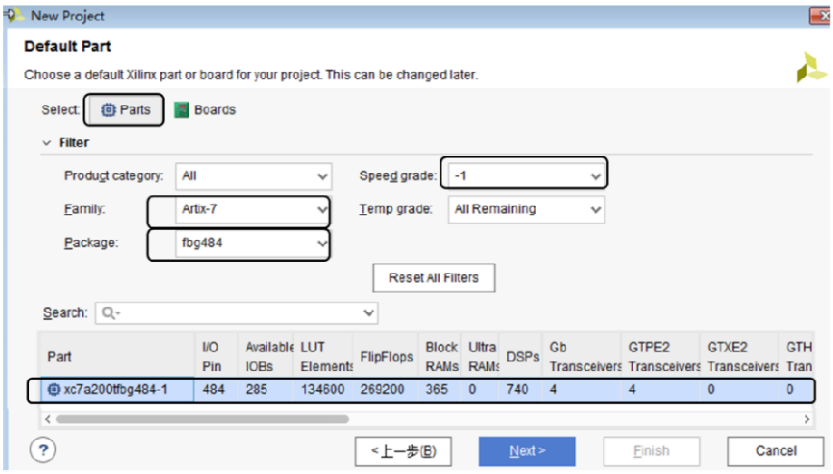
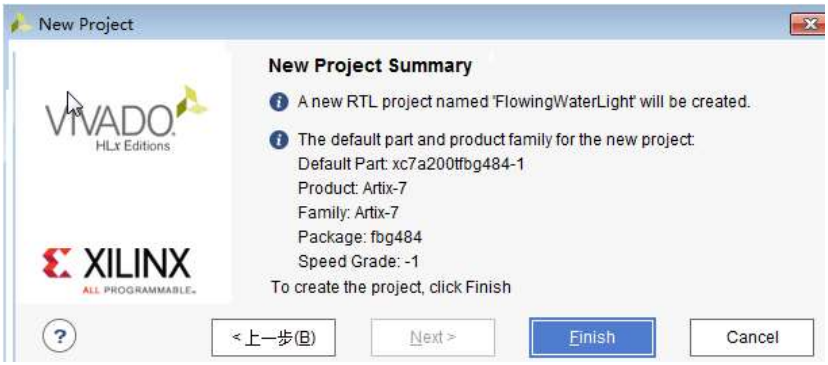


图 4.24　Vivado 新建工程向导之四

（6）在图 4.25 中，确认相关信息与设计所用的的 FPGA 器件信息是否一致，一致请点击 Finish，完成空白工程的创建，如图 4.26 所示。



4.25　Vivado 新建工程向导之五

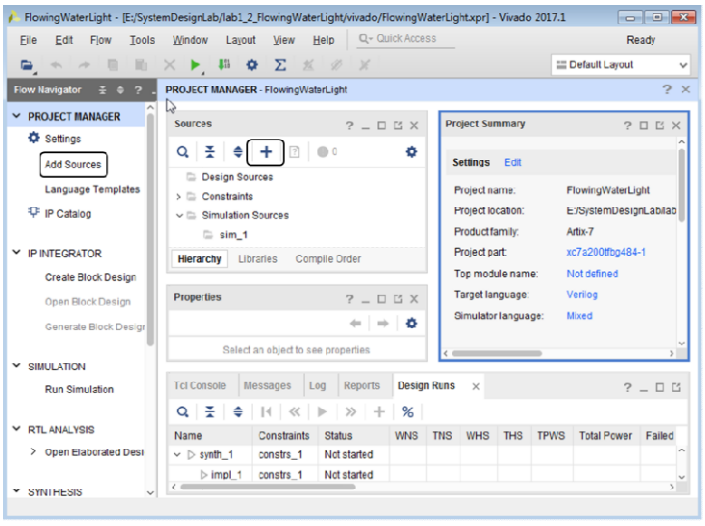


图 4.26 新建工程完成后的界面

2．加入设计文件

（1）在图 4.26 所示工程界面，在 Flow Navigator 窗口中，点击 PROJECT MANAGER⇨Settings⇨Add Sources 命令，打开设计文件导入添加对话框，如图 4.27 所示。选择第二项 Add or Create Design Sources，用来添加或新建 Verilog 或 VHDL 源文件，点击 Next，进入图 4.28 所示界面。

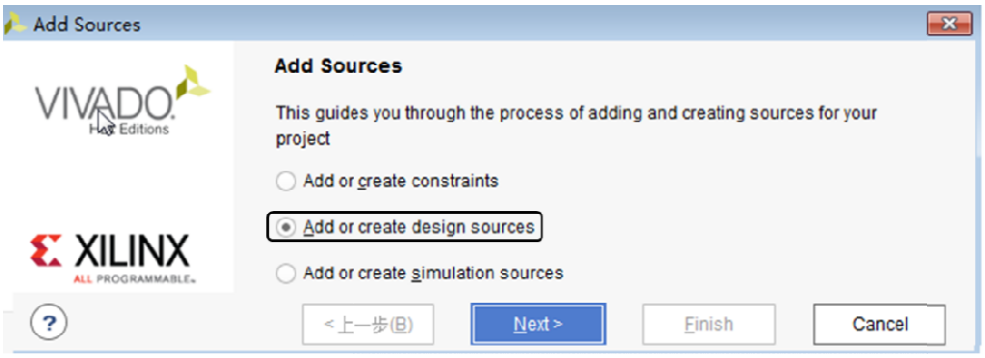


图 4.27 设计文件导入添加对话框

（2）在图 4.28 界面，通过点击 Add Files 按钮，根据添加文件导航选择需加入的设计文件。本例需要加入 FlowingWaterLight.v、~~dff.v、~~counter_n.v、 counter_up_down.v 和 decode.v 共 4 个文件。这些文件均存在文件夹 E:\SystemDesignLab\lab1_2_FlowingWaterLight\src 下。注意，每次可选择多个文件。

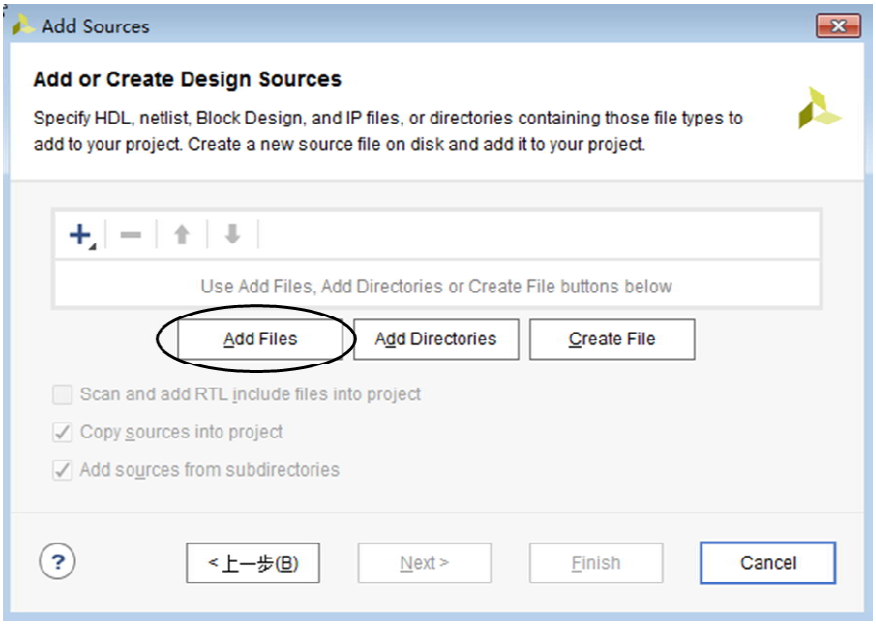


图 4.28 导入设计文件对话框之一

（3）选择添加文件后的界面如图 4.29 所示，点击 Finish 按钮，即可完成文件的导入。因为有两个开发软件 ModelSim 和 Vivado 都可能修改设计文件，必须将代码保留在 src 文件夹里，所以图 4.29 界面中的 Copy Sources into project 这一选项千万不要勾选。

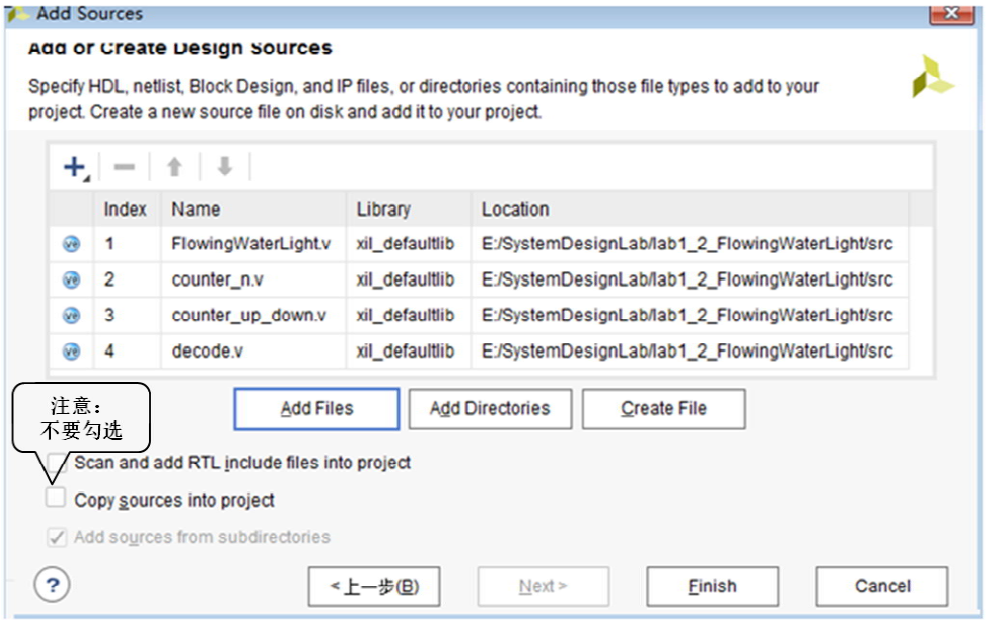


图 4.29 导入设计文件对话框之二

导入文件后，Sources 窗口列出设计文件层次结构，如图 4.30 所示。

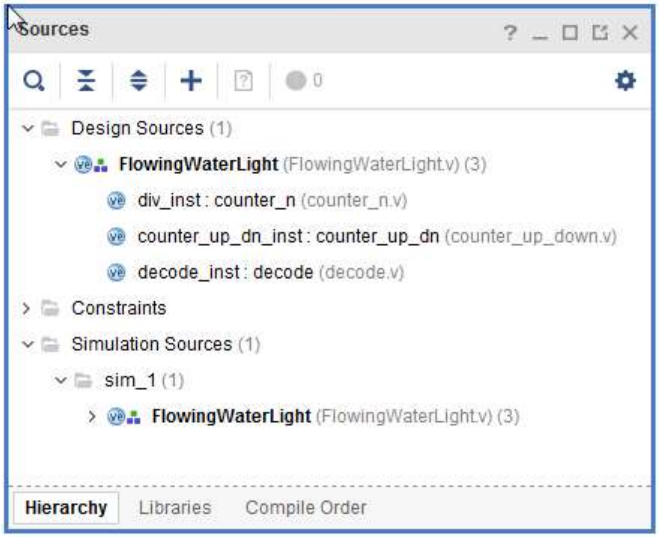


图 4.30 设计文件的层次结构

若文件层次结构不符设计要求，说明代码有错误，应改正。因为设计代码已经过 ModelSim 仿真，所以出现语法和模块调用错误的概率较少。

小提示：在 Sources 窗口双击文件，可在 Text Editor 窗口查看和编辑设计文件。若有语法错误，相应语句会有红色波浪下划线标示。

3．电路代码综合

在 Flow Navigator 窗口下，点击 SYNTHESIS ⇨Run Synthesis 命令，Vivado 将对工程进行综合。综合完成之后，在弹出的如图 4.31 所示的窗口中选择 Open Synthesized Design 选项并

点击 OK 按钮，打开综合结果。

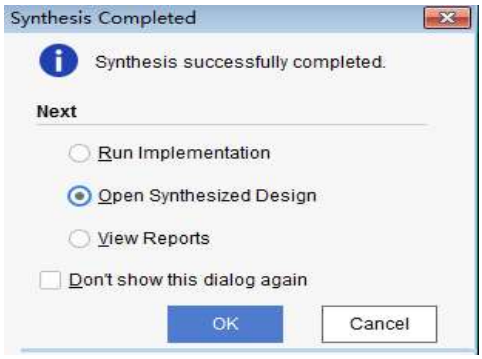


图 4.31　综合完成后界面

对初学者来说，综合结果中的原理图较为重要。可在图 4.32 的 Layout 下列表框选择 Debug 选项打开综合后的原理图。

小提示：在原理图，点击模块左上角的“+”可展开该模块的底层设计，同样，点击“-”收起底层设计。

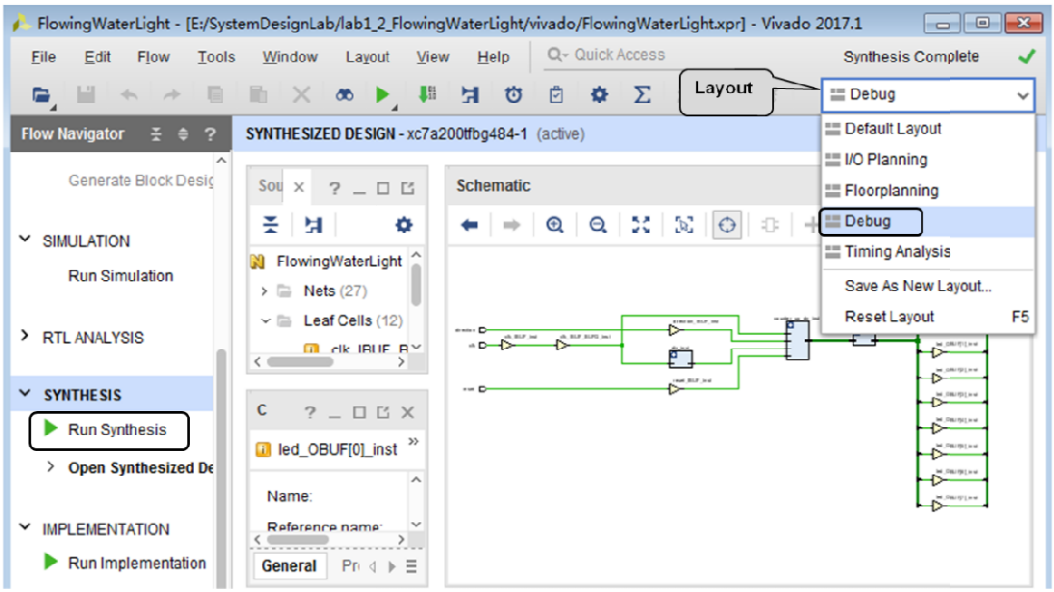


图 4.32 综合后的原理图

原理图有助于查看模块之间的连接，也可检查 HDL 语言描述的电路是否符合设计要求，因此，原理图是查找设计错误的好帮手。

4．约束

约束可以分为时钟约束、I/O 约束以及时序例外约束三大类，约束目的是：提高设计的工作频率；获得正确的时序分析报告；指定 FPGA 引脚位置与电气标准。由于篇幅限制，本例只对引脚分配和主时钟的周期进行附加约束。

有两种方法可以添加约束文件，一是在图形界面进行设置；二是直接新建 XDC 约束文件，手动输入约束命令。为使初学者更好地理解电路概念，作者采用第一种方法。

（1）引脚约束

引脚分配由 FPGA 开发板来决定，表 4.1 列出两种开发板的引脚分配方法，可根据自己手上的开发板来选择。

表 4.1　FPGA 引脚约束内容

I/O 名称	引脚编号 (Nexys Video)	引脚编号 (Basys 3)	电气标准	说　　明
clk	R4	W5	LVCM OS33	系统 100MHz 主时钟信号
reset	B22	U18	LVCM OS33	中间按键（BTNC）
direction	E22	V17	LVCM OS33	逻辑开关 SW0
Led[0]	T14	U16	LVCM OS33	LED 指示灯
Led[1]	T15	E19	LVCM OS33	
Led[2]	T16	U19	LVCM OS33	
Led[3]	U16	V19	LVCM OS33	
Led[4]	V15	W18	LVCM OS33	
Led[5]	W16	U15	LVCM OS33	
Led[6]	W15	U14	LVCM OS33	
Led[7]	Y13	V14	LVCM OS33	

①如图 4.33 所示，在工程界面中 Layout 列下列表框中选择 IO planning 选项，并在下方的选项卡中切换到 I/O ports 页面，I/O ports 窗口下将罗列出设计的所有输入/输出引脚。

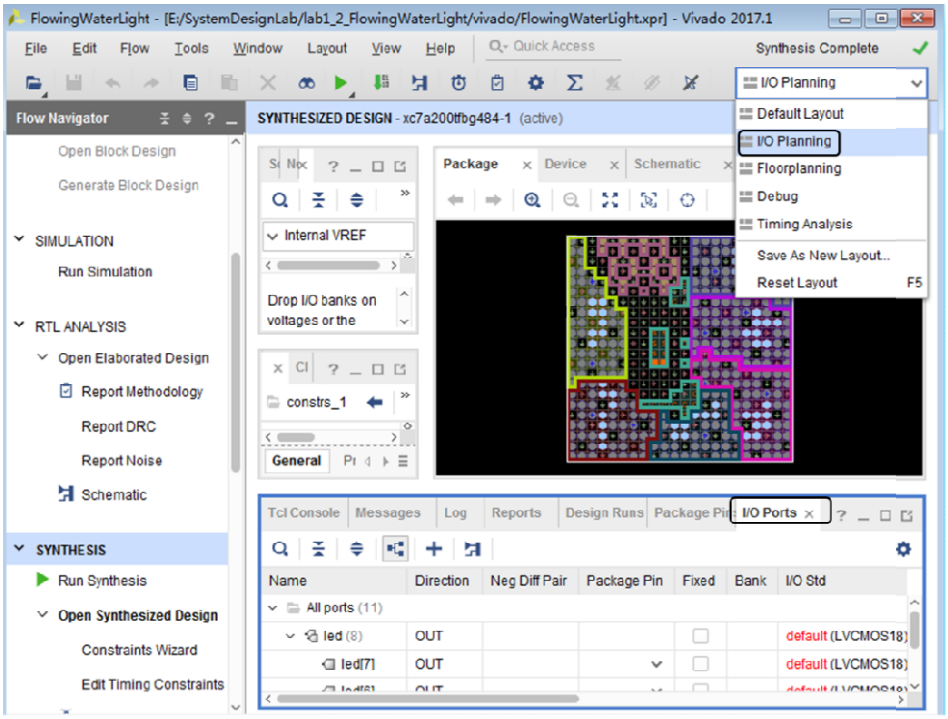


图 4.33　引脚约束之一

②如图 4.34 所示，在 I/O ports 页面，展开引脚。对于每个输入或输出引脚，在 Package Pins 栏填写或选择对应引脚编号；并在 I//O std 栏下拉选择引脚的电气标准，引脚的电气标准常采用 LVCMOS33。当引脚约束没问题时，相应 Fixed 栏将处于勾选状态。

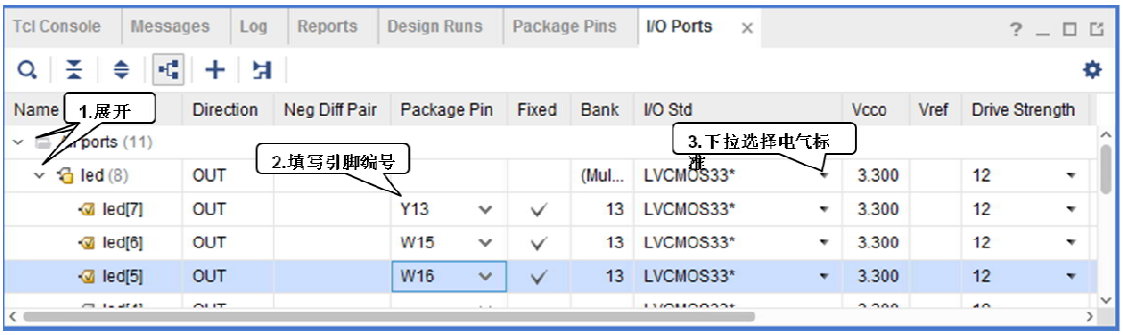


图 4.34 引脚约束之二

③完成所有引脚约束之后，点击工具栏中的保存按钮，在图 4.35 所示的界面中输入约束文件的文件名：FlowingWaterLight；选择存储路径，一般存储在源文件夹 src 中。然后，点击 OK 完成引脚约束过程。

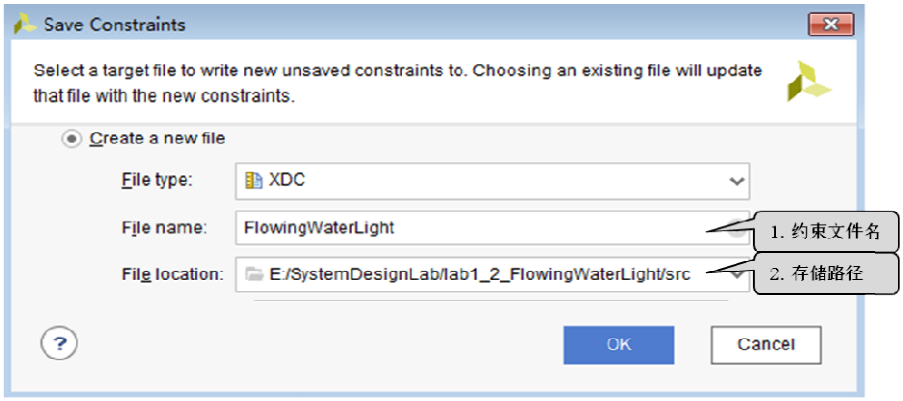


图 4.35 填写约束文件名

完成引脚约束后，在 Sources 窗口下 Constraints 中会找到的 XDC 文件，如图 4.36 所示，双击 XDC 文件，可在 Text Editor 窗口打开查看和编辑 XDC 文件内容。

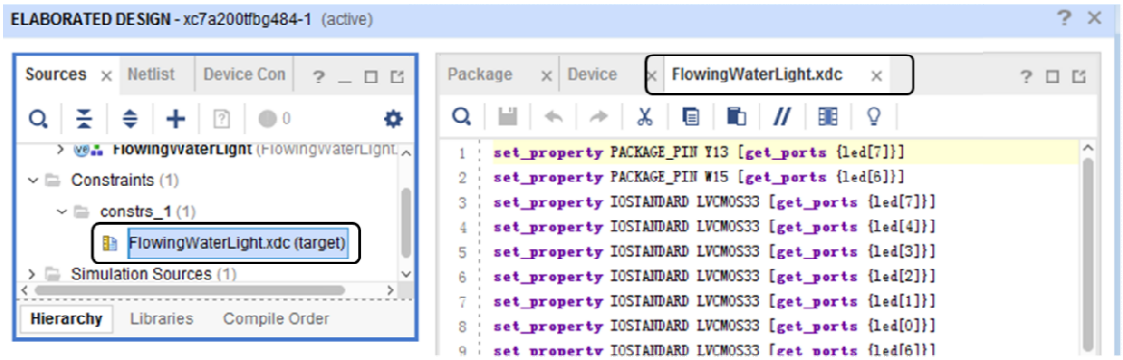


图 4.36 约束文件及其内容

（2）时钟约束

本例仅对主时钟的周期和占空比进行附加约束，开发板输入主时钟频率为 100MHz。方法如下。

在 Flow Navigator 窗口下，点击 SYNTHESIS ⇨Open Synthesis Design ⇨ Constraints ⇨Constraints Wizard 命令，在弹出的 Timing Constraints Wizard 对话框中点击 Next 按钮，然后在图 4.37 所示的界面中填写时钟频率（100MHz），由于占空比 50%，因此其它参数采用默认。点击 Skip to Finish 按钮后，再在弹出的 Timing Constraints Wizard 对话框中点击 Finish 按钮，即可完成时钟约束。

完成时钟约束后，约束文件 FlowingWaterLight.XDC 的最末添加了的一行时钟约束语句：

```
create_clock -period 10.000 -name clk -waveform {0.000 5.000} [get_ports clk]
```

小提醒：如果进行时钟约束时，没有关闭约束文件 FlowingWaterLight.XDC，此时需要 Reload 约束文件才能看到添加的时钟约束语句。

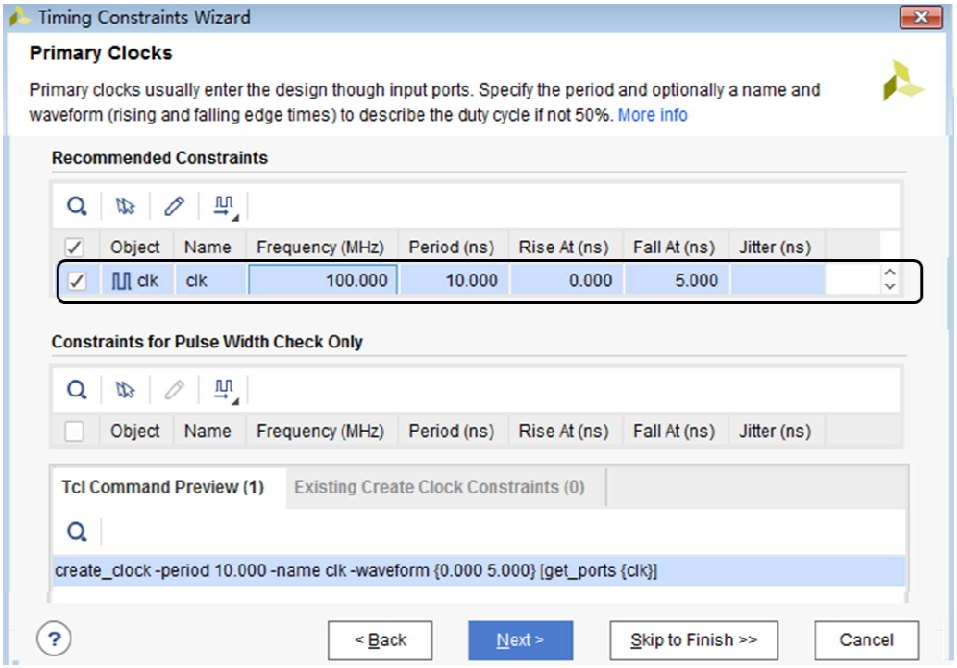


图 4.37 时钟约束

5．工程实现

设计实现是指将综合输出的网表适配到 FPGA 上，方法如下。

在 Flow Navigator 窗口下，点击 IMPLEMENTATION⇨ Run Implementaion 命令，在弹出的 Launch Runs 窗口，直接点击 OK 按钮进行实现工作。由于进行约束后没有重新综合，所以 Vivado 会依次完成综合、实现两个过程。

实现完成后，在如图 4.38 所示的界面中，可选择第二选项 Generate Bitstream，然后点击 OK 按钮，直接进行编程比特流生成工作。也可点击 Cancel 按钮取消。

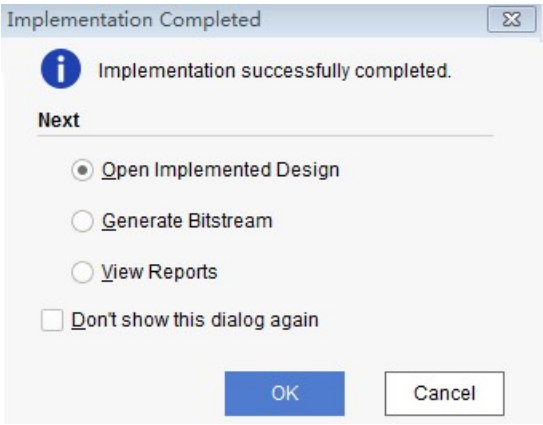


图 4.38 设计实现完成界面

6．生成 FPGA 编程比特流

FPGA 编程比特流的生成可从设计实现完成后的界面直接进入。也可在 Flow Navigator 窗口下，点击 PROGRAM AND DEBUG⇨Generate Bitstream 命令，然后在弹出的 Launch Runs 窗口，直接点击 OK 按钮的进入。

完成后，在弹出的界面也可点击 Cancel 按钮。

7．编程下载

（1）用 USB 线将开发板与计算机的 USB 接口连接。

（2)用 3.3V 的 DC 电源适配器给 Nexys Videog 开发板供电。Basys3 开发板由 USB 供电，所以，如使用 Basys3 开发板，则不需外加电源供电。

（3）打开开发板电源。

（4）在 Flow Navigator 窗口下，点击 PROGRAM AND DEBUG⇨Open Hardware Manger 命令。在图 4.39 所示的 HARDWARE MANGER 对话框中，点击 OpenTarget 后，再点击 Auto Connect 命令。Vivado 会自动查找 JTAG chain 上的设备。连接完成，FPGA 芯片会出现在 Hardware 窗口里，如图 4.40 所示。

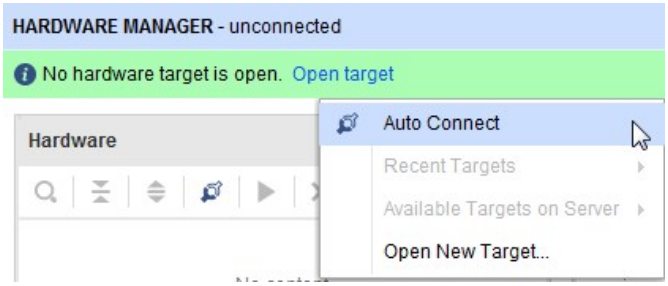


图 4.39　配置对话框之一

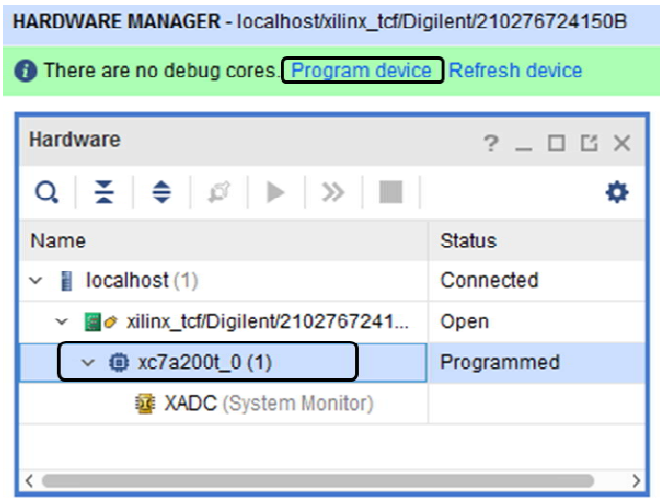


图 4.40　配置对话框之二

（5）点击图 4.40 界面中的 Program device，然后在弹出如图 4.41 所示 Program Device 窗口中点出 Program 即可完成下载编程。



图 4.41　下载

8．在实验板验证实验结果

根据 I/O 引脚约束内容，在开发实验板上拨动相应的开关、按键，观看流水灯是否符合设计要求，验证设计是否成功。

至此，整个流水灯实验完成任务。

五、思考题

（1）Vivado 的开发流程主要有哪几个步骤？各个步骤主要作用是什么？

（2）附加约束的主要目的是什么？时序约束的主要种类有哪些？

（3）Vivado 集成开发工具自带仿真器，为什么仍通常要求使用第三方仿真软件 ModelSim？

实验 3　IP 内核的使用与仿真

一、概述

1．IP 内核的基本概念

IP（Intellectual Property）内核模块是一种预先设计好的甚至已经过验证的具有某种确定功能的集成电路、器件或部件。IP 内核模块将一些在数字系统中常用但比较复杂的功能电路设计成可修改参数的模块，使其他用户可以直接调用这些模块，这样就大大减轻了用户的工作量，避免重复劳动。随着 CPLD/FPGA 的规模越来越大，设计越来越复杂，使用 IP 核是一个发展趋势。

IP 内核模块有行为（behavior）、结构（structure）和物理（physical）三级不同程度的设计，对应的 IP 内核模块也分为三大种类：软 IP 内核（soft IP core）、固 IP 内核（firm IP core）和硬 IP 内核（hard IP core）。

软 IP 内核又称为虚拟器件，通常以某种 HDL 文本形式提交给用户，它已经过行为级设计优化和功能验证，但其中不含有任何具体的物理信息。据此，用户可以综合出正确的门电路级网表，并可以进行后续结构设计，具有很大的灵活性。软 IP 内核可以很容易地借助 EDA 综合工具与其他外部逻辑电路结合成一体，根据各种不同的半导体工艺，设计成具有不同性能的器件。但是，如果后续设计不当，有可能导致整个结果失败。商品化的软 IP 内核一般电路结构总门数都在 5000 门以上。

硬 IP 内核是基于某种半导体工艺的物理设计，有固定的拓扑布局和具体工艺，并已经过工艺验证，具有可保证的性能。其提供给用户的形式是电路物理结构掩模版图和全套工艺文件，是可以拿来就用的全套技术。

固 IP 内核的设计深度则是介于软 IP 内核和硬 IP 内核之间，除完成软 IP 内核所有的设计外，还完成了门电路级综合和时序仿真等设计环节。一般以门电路级网表形式提交给用户使用。

常用的 IP 内核模块包括各种不同的 CPU（32/64 位 CISC/RISC 结构的 CPU 或 8/16 位微控制器/单片机，如 8051 等）、32/64 位 DSP（如 320C30）、DRAM、SRAM、EEPROM、Flash Memory、A/D 转换器、D/A 转换器、MPEG/JPEG 编解码器、USB 接口、PCI 标准接口、网络单元和模拟器件模块等。丰富的 IP 内核模块库为快速设计专用集成电路和单片系统及尽快占领市场提供了基本保证。

2．DCM 内核的组成和功能介绍

数字时钟管理 DCM（Data Clock Management）由 4 部分组成，如图 4.42 所示。其中，最低层采用数字延迟型锁相环（Delay Locked Loop，DLL）模块，其次分别是数字频率合成器（Digital Frequency Synthesizer，DFS）和数字移相器（Digital Phase Shifter，DPS）。不同芯片模块的 DCM 输入频率范围是不同的。

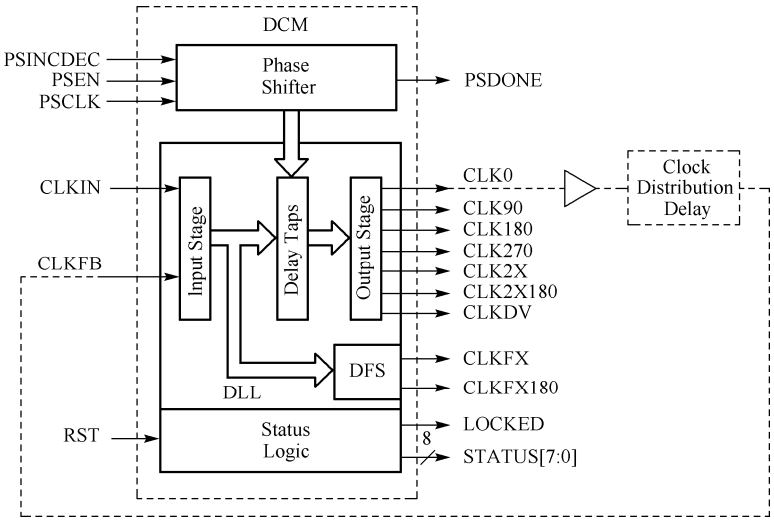


图 4.42　DCM 功能块和相应的信号

1）DLL 模块

DLL 主要由一个可变延时线和控制逻辑组成。延时线对时钟输入端 CLKIN 产生一个延时，时钟分布网线将该时钟分配到器件内的各个寄存器和时钟反馈端 CLKFB；控制逻辑在反馈时钟到达时采样输入时钟以调整二者之间的偏差，实现输入和输出的零延时，如图 4.43 所示。具体工作原理是：控制逻辑在比较输入时钟和反馈时钟的偏差后，调整延时线参数，直到输入时钟和反馈时钟的上升沿同步，使环路进入“锁定”状态，只要输入时钟不发生变化，输入时钟和反馈时钟就保持同步。DLL 可以用来实现一些完善和简化系统级设计的电路，如提供零传播延时、低时钟相位差和高级时钟区域控制电路等。DLL 引脚说明如下。

CLKIN：DLL 输入时钟信号，通常来自 IBUFG 或 BUFG。

CLKFB：DLL 时钟反馈信号，该反馈信号必须源自 CLK0 或 CLK2X，并通过 IBUFG 或 BUFG 相连。

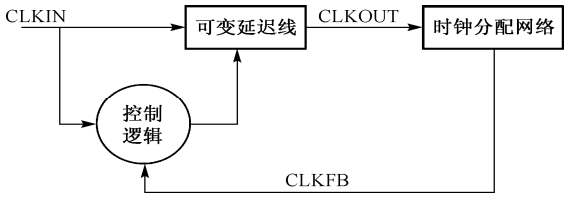


图 4.43　DLL 模型简单示意图

RST：DLL 复位信号，高电平有效。该信号控制 DLL 的初始化，通常接地。

CLK0、CLK90、CLK180、CLK270：四个同频时钟输出。其中，CLK0 与 CLKIN 无相位偏移，CLK90 与 CLKIN 有 90° 相位偏移，CLK180 与 CLKIN 有 180° 相位偏移，CLK270 与 CLKIN 有 270° 相位偏移。

CLKDV：时钟分频输出，是 CLKIN 的分频时钟信号。DLL 支持的分频系数为 1.5、2、2.5、3、4、5、8 和 16。

CLK2X、CLK2X180：CLKIN 的 2 倍频时钟信号输出。

LOCKED：锁定指示。当 DLL 完成锁存之后，LOCKED 有效（高电平）。

在 FPGA 设计中，消除时钟的传输延迟、实现高扇出最简单的方法就是利用 DLL，把 CLK0 与 CLKFB 相连即可。利用一个 DLL 还可以实现 2 倍频输出，如图 4.44 所示。

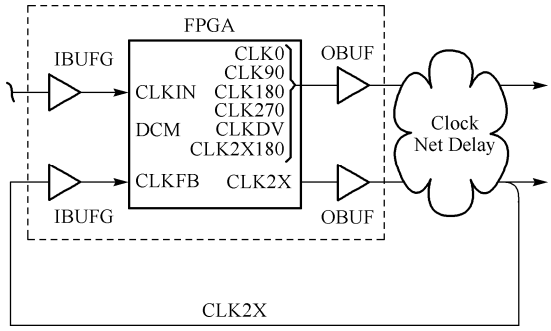


图 4.44　Xilinx DLL 2 倍频典型模型示意图

2）数字频率合成器（DFS）

DFS 可以为系统产生丰富的频率合成时钟信号，能提供输入时钟频率分数倍或整数倍的时钟输出频率方案，输出信号为 CLKFX 和 CLKFX180，输出频率范围为 1.5～320MHz（不同芯片的输出频率范围是不同的）。这些频率基于用户自定义的两个整数比值，一个是乘因子 M，另外一个是除因子 D，输入频率和输出频率之间的关系为

$$F_{\mathrm{CLKFX}} = F_{\mathrm{CLKIN}} \times \frac{M}{D} \tag{4.1}$$

3）数字移相器（DPS）

通过 DPS，DCM 具有移动时钟信号相位的能力，因此能够调整 I/O 信号的建立和保持时间，能支持对其输出时钟进行 0°、90°、180°、270° 的相移粗调和相移细调。其中，相移细调对相位的控制可以达到输入时钟周期的 1%的精度（或者 50ps），并且具有补偿电压和温度漂移的动态相位调节能力。对 DCM 输出时钟的相位调整需要通过属性控制 PHASE_SHIFT 来设置。PHASE_SHIFT 的设置范围为–255～+255。比如，输入时钟为 200MHz，需要将输出时钟调整+0.9ns，则 PHASE_SHIFT = (0.9ns/5ns) ×256 = 46。如果 PHASE_SHIFT 值是一个负数，表示时钟输出应该相对于 CLKIN 向后进行相位移动；如果 PHASE_SHIFT 是一个正数，则表示时钟输出应该相对于 CLKIN 向前进行相位移动。

移相用法的原理图与倍频用法的原理图很类似，只需把 CLK2X 输出端的输出缓存移到 CLK90、CLK180 或 CLK270 端即可。利用原时钟和移相时钟与计数器相配合也可以产生相应的倍频。

DCM 主要功能有。

1）分频倍频：DCM 可以将输入时钟进行倍频或分频，从而得到新的输出时钟。

2）消除时钟抖动：DCM 可以消除时钟抖动，所谓时钟抖动就是由于传输延迟，引起时钟到达不同地点的时间差。

3）相移：DCM 可以实现对输入时钟的相移输出，这个相移一般是时钟周期的一个分数。

4）全局时钟：DCM 和 FPGA 内部的全局时钟分配网络紧密结合，因此性能优异。

5）电平转换：通过 DCM，可以输出不同电平标准的时钟。

二、实验设备

（1）装有 Vivado、ModelSim SE 软件的计算机。

（2）Nexys Video Artix-7 FPGA 多媒体音视频智能互联开发系统。

（3）具有 HDMI 接口显示器一台，也可用“HDMI-VGA 转换器+VGA 显示器”替代。

三、实验要求

本实验将在显示器上显示一幅 128×128 像素的彩色轴对称图片，即图片上下、左右均对称。系统原理框图如图 4.45 所示。图中的~~输入~~输入 clk 为 100 MHz，时钟管理模块 DCM 产生 25MHz 的像素时钟 pixel_clk 和 250MHz 的 HDMI 位时钟 tmds_clk。PictureROM 中存放一幅 64×64 像素的彩色图片（轴对称图片的四分之一），图片格式为 24bits 位图，因此 ROM 容量为 2^{12}×24bits。

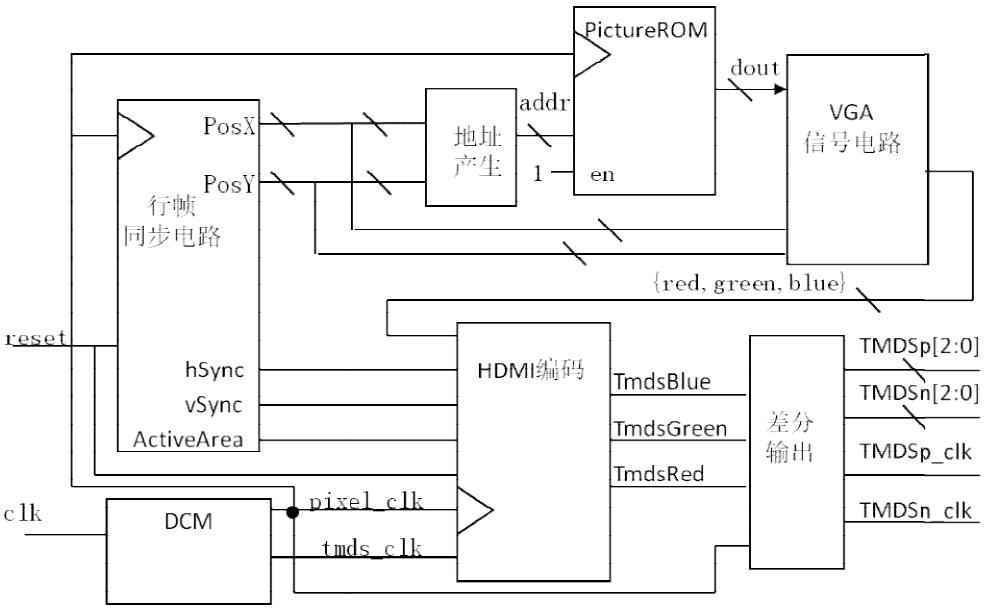


图 4.45　系统原理框图

本实验要求用 Xilinx 内核生成器系统生成一个 DCM 模块和一个 2^{12}×24bits 的 ROM 模块，并将 DCM 内核和 ROM 内核应用到设计中。

四、实验步骤

1.建立 Vivado 工程

（1）从网站中下载 lab3_IPCore 文件夹至硬盘里，存放在 e:\SystemDesignLab 文件下。lab3_IPCore 共五个子文件夹，其中 src、sim 和 vivado 三个文件夹分别存放源代码、ModelSim 工程和 Vivado 工程；pic 文件夹存放了一张一幅 64×64 像素的彩色图片；而 tool 文件夹有一

个小应用软件 coe3.0.exe，该软件可将位图生成 ROM 初始化 coe 文件。

（2）新建 Vivado 工程，工程名为 IPCore，工程存放在 e:\SystemDesign\Lab\Vivado 文件夹，添加下列~~设计~~文件。

①设计文件:顶层设计模块 IPCore.v、行帧同步电路模块 syncGenarator.v、HDMI 编码模块 TMDSencode.edf 和 TMDSencode.v。

注意， HDMI 编码模块以 BlackBox 方式提供，即提供综合网表文件 TMDSencode.edf 和端口文件 TMDSencode.v。某种意义上，HDMI 编码模块是作者以软核方式提供给读者。

②用户约束文件 IPCore.xdc。

上述文件均存放在 src 子文件夹，添加设计文件后的 Vivado 工程的 Sources 窗口如图 4.46 所示。从文件结构中可看出，工程还缺少 DCM_PLL 和 PictureROM 两个模块，这两个模块将由 IP 内核生成，下面将分别介绍这两个模块生成和应用。

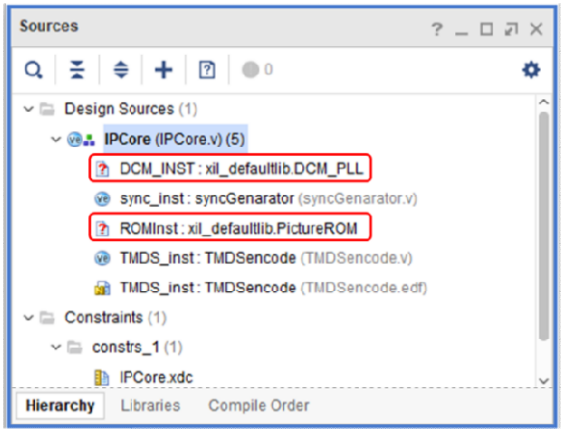


图 4.46　IP 核的 ISE 工程界面

2．ROM 内核的生成和应用

（1）准备工作，双击运行 e:\SystemDesignLab\tool 文件夹中的 coe3.0.exe 程序，界面如图 4.47 所示。在图 4.47 中点击“打开 bmp 文件”按钮，在弹出的界面，选择 e:\SystemDesignLab\fig\fig.bmp 文件，并点击“打开”按钮。最后， 点击图 4.47 中“生成 coe 文件”按钮，即可在存放图片 fig 文件夹里生成 ROM 初始化文件 fig.coe。



图 4.47　ROM 初始化 coe 文件的生成

（2）在图 4.48 所示的 Vivado 工程界面中，在 Flow Navigator 窗口下，点击 PROJECT MANAGER⇨ IP Catalog 命令；然后在 Text Editor 窗口的 IP Catalog 选项卡中，找到并依次展开 Memories & Storage Elements、RAMs & ROMs；双击 RAMs & ROMs 下面的 Distributed Memory Genertor 选项，将会弹出如图 4.49 所示的存储器 IP 内核产生向导。

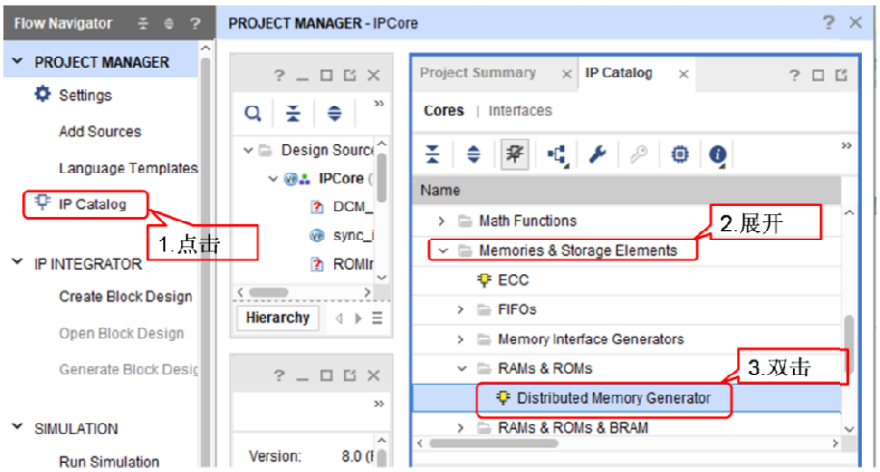


图 4.48　选择 IP 对话框

（3）在图 4.49 中配置 IP 内核存储器的模块名、容量和类型等参数。本例具体参数如下。

①模块名（Component Name）：PictureROM。注意，一旦生成了 IP 核，就算打开 IP 核进行修改，也不能再更改这个 IP 核的模块名了，所以取名需谨慎。

②存储器容量：字数（Depth）为 4096，存储器位数（Width）为 24 位。

③存储器类型选 ROM。

配置完毕后，单击 Port config 选项卡进入存储器的端口特性配置界面，如图 4.50 所示。

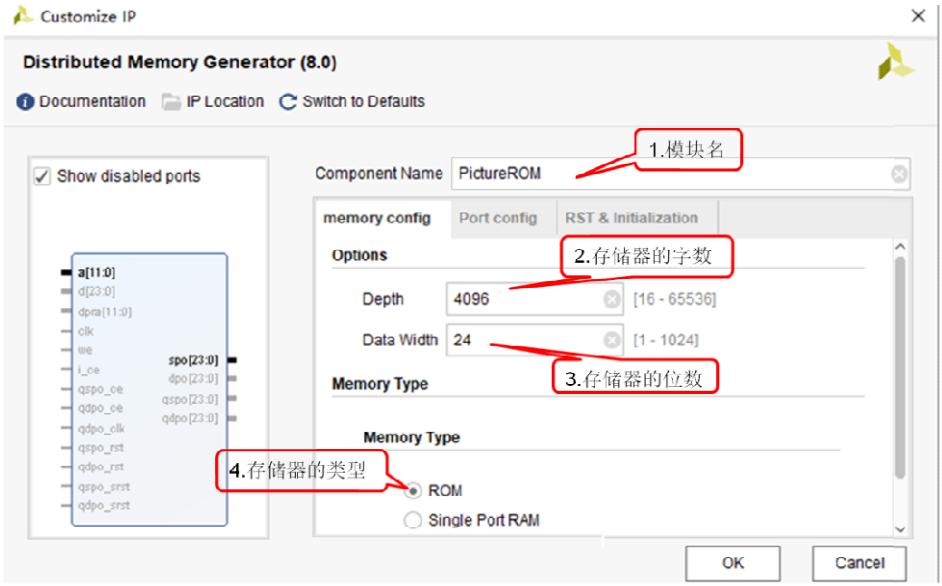


图 4.49　存储器配置界面之一

（4）在图 4.50 所示配置界面主要用于配置引脚特性，本例输入引脚采用默认配置。而输出引脚采用寄存器输出方式，并有输出使能控制。即在 Output Option 栏中选择 Registered,并勾选 Single Port Output CE 复选框。

配置完毕后，单击 RST & Initialization 选项卡进入存储器的初始化配置界面，如图 4.51 所示。

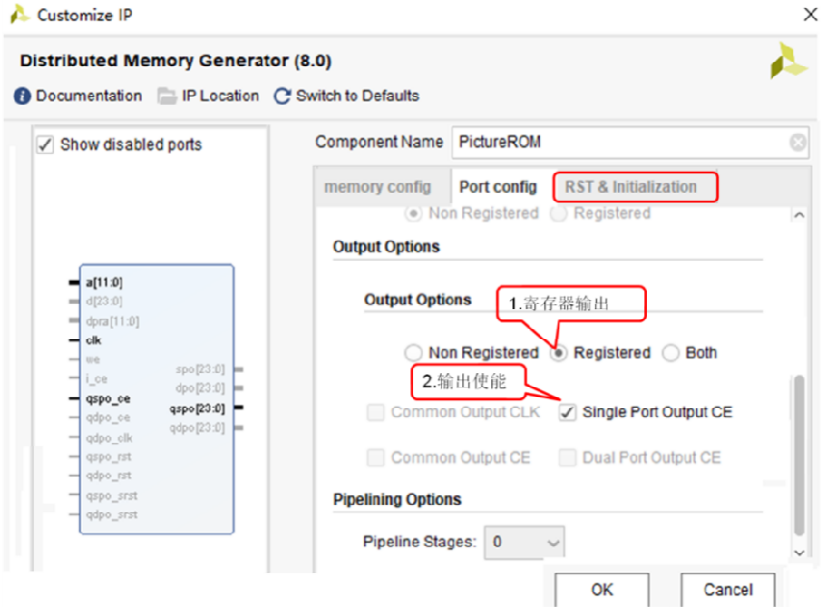


图 4.50　存储器配置界面之二

（5）图 4.51 所示对话框用来初始化存储器，即配置 ROM 内容或 RAM 存储器初值。本例为 ROM， ROM 初始化文件 fig.coe。因此，只需在 Load COE File 栏中，点击文件打开按钮，选取 e:\SystemDesignLab\fig\fig.coe 文件即可。

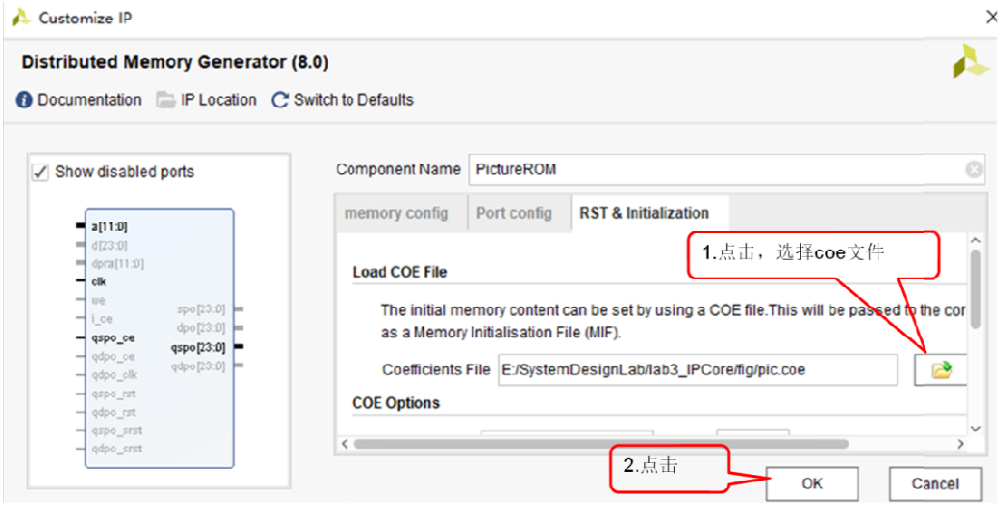


图 4.51　存储器配置界面之三

至此，存储器配置完毕。点击 OK 按钮，在可能弹出的界面中点击 OK 按钮进入存储器 IP 内核生成的最后一个界面，如图 4.52 所示。在此对话框中，选择综合方式 Out of context per IP，单击底部的 Generate 按钮，Vivado 将生成 ROM 内核，并将 PictureROM.xci 文件自动加到 ISE 工程中,如图 4.53 所示。

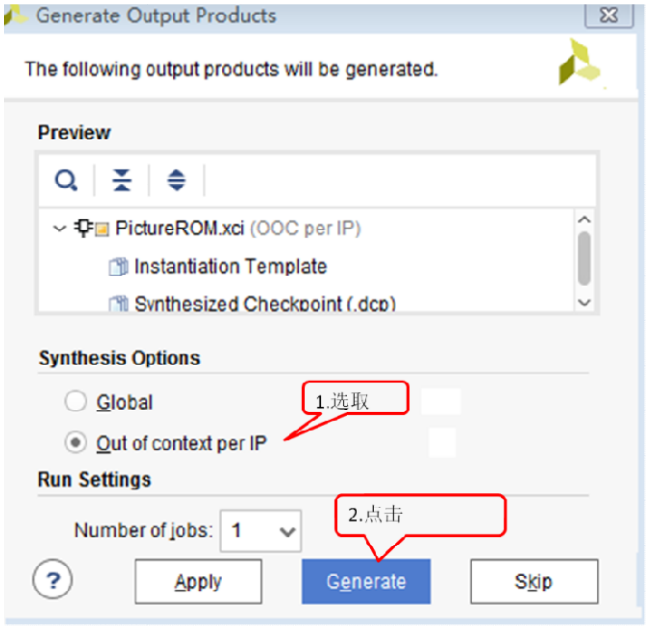


图 4.52　存储器 IP 内核生成界面

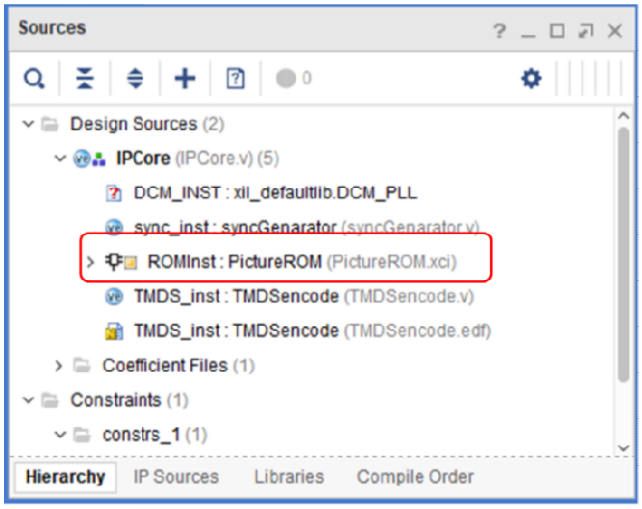


图 4.53　存储器 IP 内核生成后 Sources 窗口

存储器 IP 内核生成后，除了用于综合和实现的 xci 文件后，还产生了用于不同的应用的大量文件。如图 4.54 所示，点击 IP Sources 选项卡，Sources 窗口中罗列出 IP 内核的各种应用文件。例如，用于仿真的 Verilog HDL 文件 PictureROM_sim_netlist.v 和 ROM 数据交换文件 PictureROM.mif 等。这些文件均存放在 E:\SystemDesignLab\lab3_IPCore\Vivado\ IPCore.srcs \sources_1\ip\PictureROM 文件夹里。

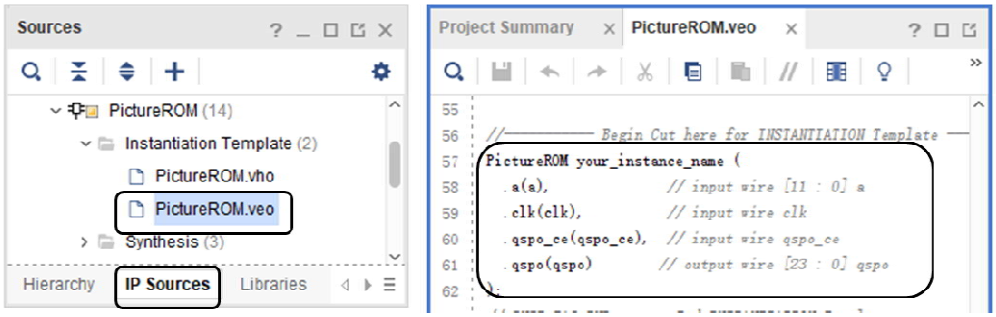


图 4.54 存储器 IP 内核文件结构

另外，如图 4.54 所示，上面文件夹中还有一个实例调用模版文件 PictureROM_.veo，双击，可在 Text Editor 窗口查看实例调用模版，便于使用者调用该 IP 内核模块。根据此模版和工程设计需求，本实验在顶层文件 IPCore.v 中添加了如下的 IP 内核模块实例。

```
//PictureROM 实例
PictureROM     ROMInst (
            .a(addr),
            .clk(pixel_clk),
            .qspo( {Picture_red,Picture_blue,Picture_green}),
            .qspo_ce(1'b1)     );
```

３．DCM 内核的生成和应用

本实验输入时钟频率为 100MHz，要求 DCM 模块产生两种时钟：25MHz 像素时钟和 250MHz 的 HDMI 位发送时钟。因此，需用 DCM 锁相频率合成技术实现。具体步骤如下。

（1）在 Vivado 工程界面中，如图 4.55 所示，在 Flow Navigator 窗口下，点击 PROJECT MANAGER⇨IP Catalog 命令。然后在在 Text Editor 窗口的 IP Catalog 选项卡中，找到并依次展开 FPGA Features and Design、Clocking，并且双击 Clocking Wizard 选项，将会弹出如图 4.56 所示的 DCM 内核生成向导。

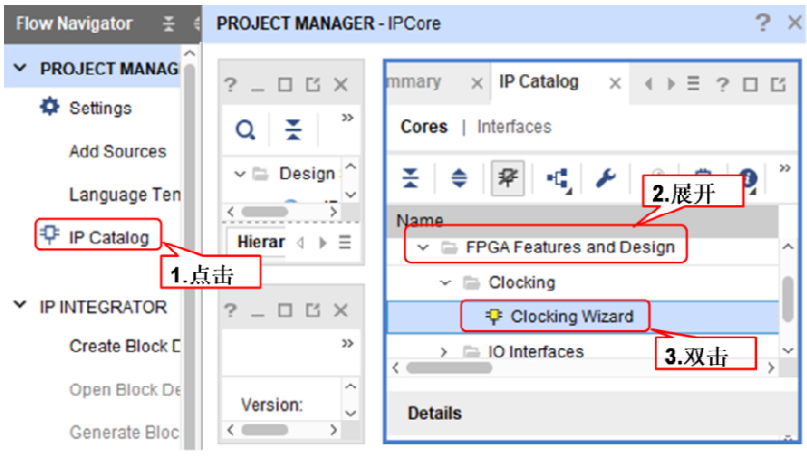


图 4.55 选择 IP 对话框

（2）在图 4.56 所示界面中，首先填写模块名：DCM_PLL，然后根据设计要求在 Clocking Options 选项卡中进行如下设置。

①时钟资源：PLL。

②输入 DCM 的输入时钟频率：100MHz。

③其它参数的采用默认配置。

设置完毕后，点击 Output Clocks 选项卡，进入 Output Clocks 页面，如图 4.57 所示。

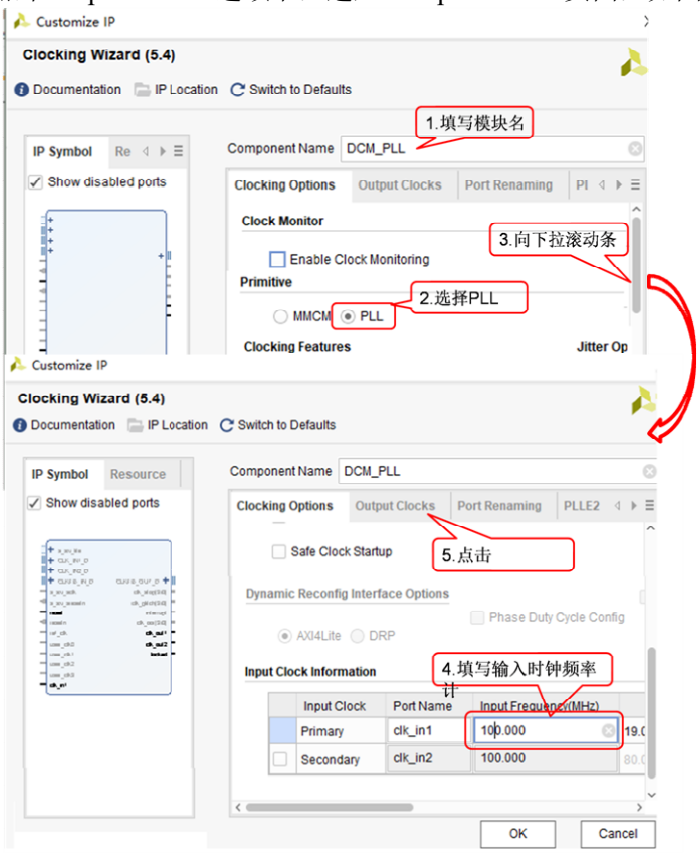


图 4.56　DCM 配置参数配置之一

（3）在图 4.57 所示 Output Clocks 选项卡中，根据设计要求对输出时钟进行如下设置。

①DCM 的输出时钟频率：clk_out1：25MHz；clk_out2：250MHz。

②勾选复位输入 reset，选择 Activ High，使复位输入高电平有效。

③其它参数的采用默认配置。

至此，DCM 配置完毕。点击 OK 按钮，在弹出对话框中，选择综合方式 Out of context per IP，单击底部的 Generate 按钮，Vivado 自动生成 DCM 内核，并将 DCM_PLL.xci 文件自动加到 Vivado 工程中。

另外，也会生成用于仿真的文件 DCM_PLL_sim_netlist.v，存放在 E:\SystemDesignLab \lab3_IPCore\Vivado\ IPCore.srcs \sources_1\ip\DCM_PLL 文件夹里。

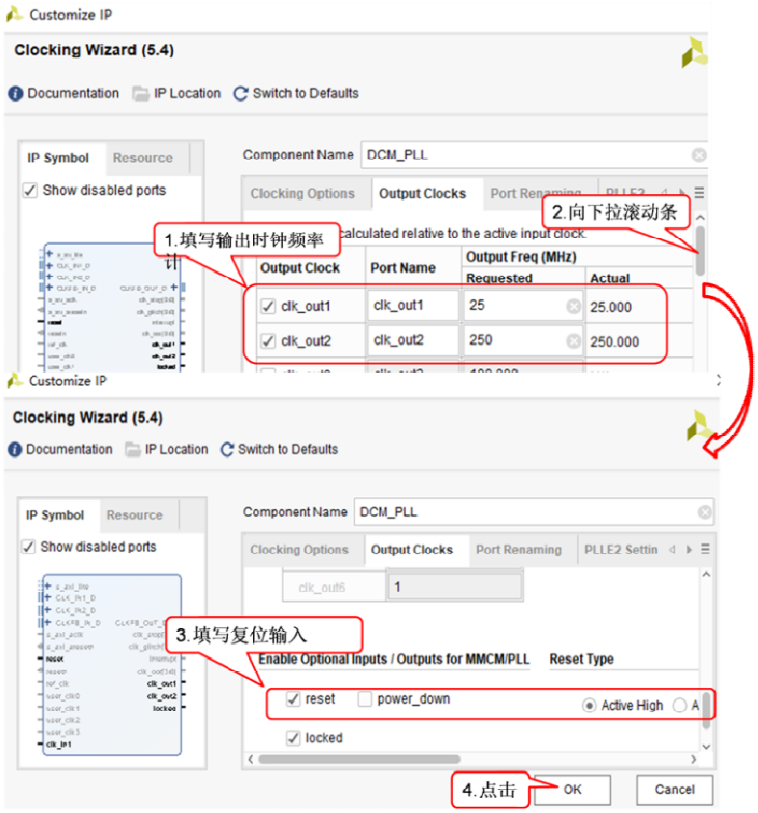


图 4.57　DCM 配置参数配置之二

3．IP 内核的仿真

用 ModelSim 软件独立仿真 Vivado 生成的 IP 核，需要用到 Xilinx 仿真库，仿真库的生成和 ModelSim 仿真环境的建立详见附录 D。

实验工程涉及视频仿真，仿真测试文件的编写思路为读取一帧图像的红、绿和蓝三基色数据，并以 RGB24 格式存储于 rgb.rgb24 文件中，最后用 RGB 播放器进行播放验证。测试文件已在第三章已作介绍，对于初学者来说，该测试文件可能较难理解，所以读者可在完成实验 12 后再来进一步理解该测试文件。

由于涉及 DCM 仿真，一般要求仿真精度达到 1ps，否则可能造成延时型锁相环失锁现象。

下面详细介绍仿真过程。

（1）仿真环境的建立，该步骤见附录 D 介绍。注意，如你在此之前已进行仿真环境的建立，这一步可略过。

（2）新建一个 ModelSim 工程，工程名 IPCoreSim，工程存放在 e:\SystemDesignLab \lab3\sim 文件夹。

（2）给工程添加下列仿真需要的文件。

① 测试文件 IPCore _tb.v。

② 顶层模块 IPcore.v。

③行帧同步控制模块 syncGenarator.v。

④HDMI 编码模块的行为模型 TMDSencode_func_synth.v。

⑤DCM 内核模块的行为模型 DCM_PLL_sim_netlist.v。

⑥ROM 内核模块的行为模型 PictureROM_sim_netlist.v。

在此强调一下，IP 内核用于综合、实现和仿真的文件是不同的。

（3）复制 ROM 内核模块的数据交换文件 PictureROM.mif 至 ModelSim 工程文件夹 e:\SystemDesignLab\lab3\sim。

（4）编译所有文件。

（5）仿真配置。

①菜单 Project⇨Add to Project⇨ Simulate Configuration 命令，弹出如图 4.58 所示的对话框。在此对话框中，首先对此仿真配置取名，名称自拟；其次单击 Libraries 选项卡，再单击 Add 按钮，在 Search Libraries 栏中添加已经编译好的仿真库，本实验需添加 simprims_ver 和 unisim_ver 二个仿真库。

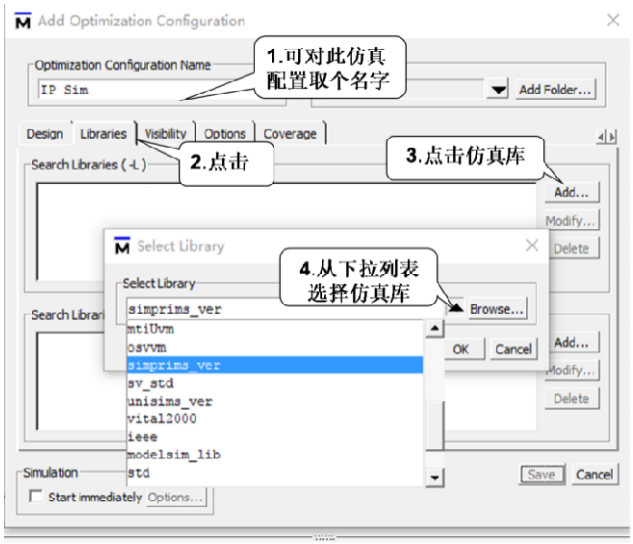


图 4.58 仿真配置之一

②如图 4.59 所示，单击 Design 选项卡，再单击 Optimization Options 按钮，在弹出的对话框中选择 Apply full visibility to all modules 选项，最后点击 OK 按钮。

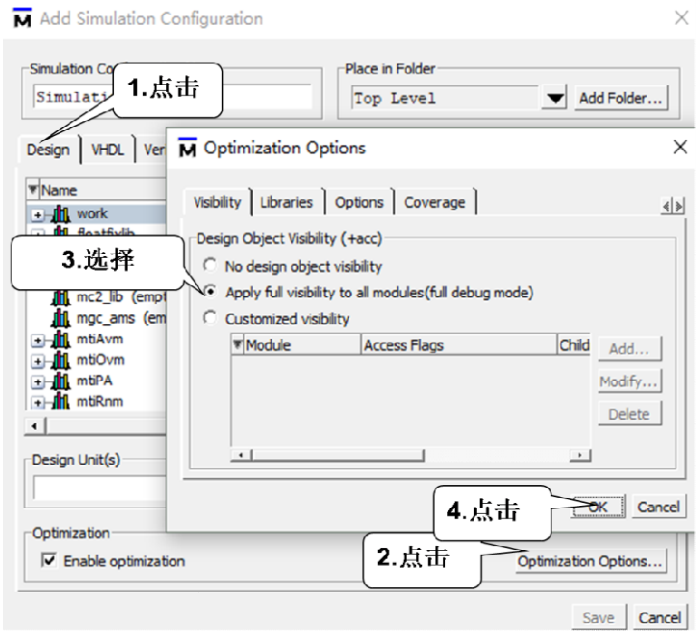


图 4.59　仿真配置之二

③如图 4.60 所示，单击 Design 选项卡， 展开 work，点击选中要仿真的文件 IPCore_tb，此时在 Design Unit(s)栏中会自动加入“work.IPCore _tb”。最后点击 Save 按钮完成仿真配置。仿真配置完成后，在 Workspace 窗口增加了一个仿真配置选项 IPSim，如图 4.61。

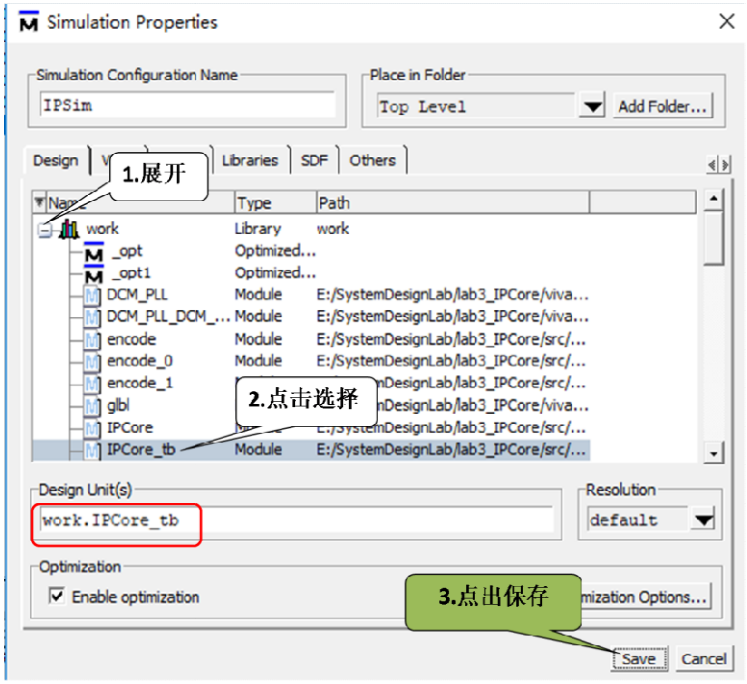


图 4.60　仿真配置之三

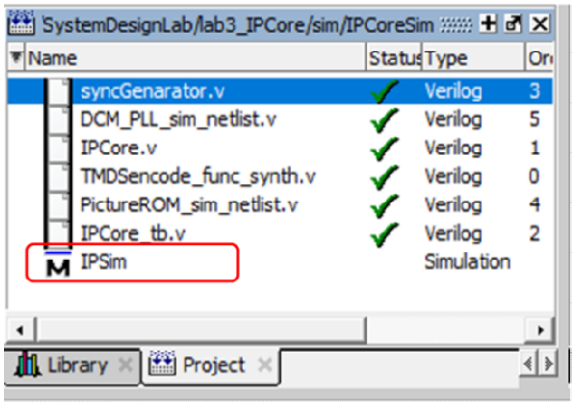


图 4.61　仿真配置

（6）完成仿真。

双击图 4.61 中仿真配置文件 IPSim，即可启动仿真，然后，添加波形，执行仿真（run all）。由于仿真精度高，且需仿真一帧完整图像，因此仿真需较长时间，约十多分钟。仿真结束，除了生成波形外，还将生成一帧视频文件 rgb.rgb24，存储在仿真工程 sim 文件夹里。

（7）查看仿真结果。

① 查看波形。

② 用播放器播放 rgb.rgb24。

运行 RawPlayer，执行菜单“文件⇨打开…”，在图 4.62 对话框中选择或填写下列信息：文件类型-rgb24，视频格式-640×480。然后选择并打开 rgb.rgb24。将会在屏幕中间看到一幅彩色图片，如图 4.63 所示。

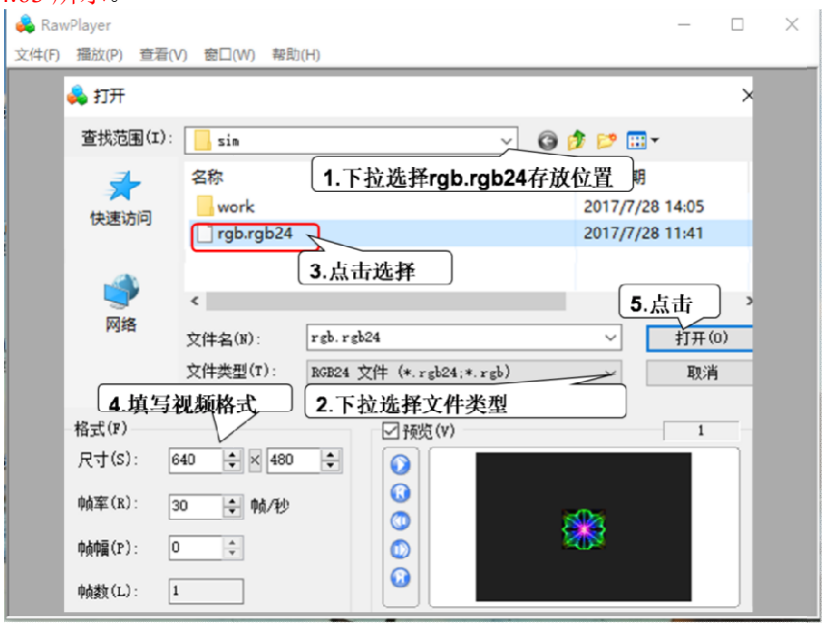


图 4.62　用 RawPlayer 软件打开视频方法

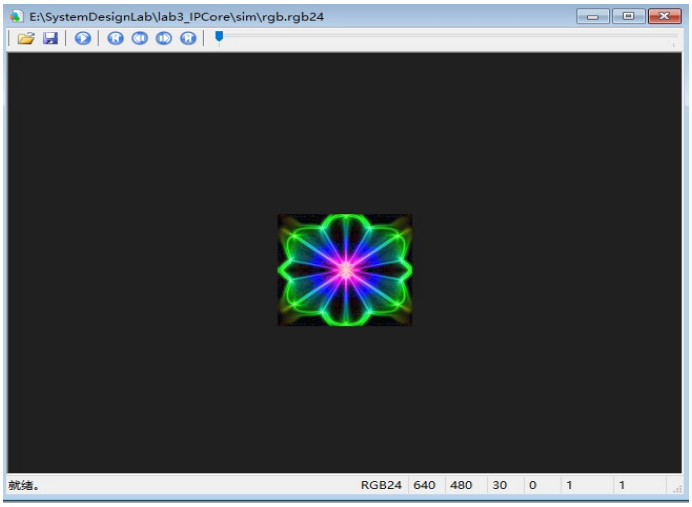


图 4.63 仿真视频结果

4．下载工程至开发板

仿真正确后，回到 Vivado 软件，对工程综合、实现、生成编程代码，最后下载至 Nexys Video Artix-7 FPGA 多媒体音视频智能互联开发系统。接入带 HDMI 接口的显示器，或通过 HDMI-VGA 转换器接入 VGA 显示器。显示器中间会显示一幅 128×128 像素彩色图片。

至此，实验操作完成。

五、思考题

（1）什么是 IP 内核？IP 内核模块分哪几类？为什么要使用 IP 内核模块？
（2）Xilinx DCM 内核主要由哪几个部分组成？各部分作用是什么？
（3）对具有 IP 内核的系统进行仿真，为什么在进行仿真前需要对厂家的仿真库进行预编译？
（4）如何在 ModelSim 中指定 Xilinx 的仿真库？

实验 4　ILA 的逻辑分析仪实验

一、概述

在 FPGA 开发中，当我们写完代码，进行仿真，验证设计没有问题后，下载到 FPGA 芯片上，一般都能按照我们的设计意愿执行相应功能。但这也并非绝对的，有时候你会遇到一些突然情况，比如时序问题或者仿真时没有考虑到某种情况，但实际中它确实存在的，这就会造成功能上的错误了。也有些时候你的设计似乎没法进行仿真或者仿真做起来很困难，比如设计一个 SDRAM 或者 DDR 控制器，需要相应写一个 SDRAM 或者 DDR 之类的模型进行仿真，这时候就很为难了。所以在很多时候，一个 FPGA 工程师应该学会使用逻辑分析仪进行分析调试。

传统的测试工具为逻辑分析仪，如图 4.64 所示，只能从 FPGA 外部去观察信号，而 FPGA 综合出来的电路都在芯片内部，因此，整个调试过程可观性非常差，故障的复现、定位、验证非常困难，严重影响开发进度。所以 FPGA 厂家就发明了内置的逻辑分析仪，在 Vivado 中叫 ILA（Integrated Logic Analyzer，集成逻辑分析仪）。其基本原理相当于在 FPGA 内部插入集成逻辑分析仪的 IP 核，利用 FPGA 中尚未使用的逻辑资源和 RAM 资源，搭建一个逻辑分析仪， ILA 内核伸出许多 probe 去探测信号线，如图 4.65 所示。然后，根据用户设定的触发条件，采集需要的内部信号并保存到 RAM，通过 JTAG 口传送到计算机，最后在计算机屏幕上显示出时序波形。

ILA 使 FPGA 不再是黑匣子，可以观察 FPGA 内部信号，对 FPGA 的内部调试非常方便，因此得到了广泛应用。



图 4.64　传统的测试方法

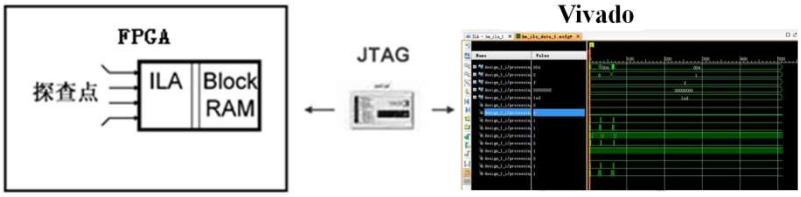


图 4.65　ILA 的应用框图

二、实验要求

本实验的工程范例为流水灯电路，与实验 1 相同，原理框图见图 4.2 所示，要求用 ILA 分析 FPGA 内部的分频器的状态信号 div_q [25:0]和可逆计数器状态 q [3:0]。

三、实验设备

（1）装有 Vivado、ModelSim SE 软件的计算机。

（2）Nexys Video 开发板或 Basys3 开发板。

四、实验步骤

1．下载文件

从网站中下载 la4_ILA 文件夹至硬盘里，存放在 e:\SystemDesignLab 文件下。子文件夹 src 有顶层 FlowingWaterLight.v、任意进制计数器/分频器 counter_ n.v、任意位数可逆计数器 counter_up_down.v、译码器模块 decode.v 和约束文件 FlowingWaterLight.xdc。

2．新建 Vivado 工程

新建一个 Vivado 工程，工程存储路径：E:/SystemDesignLab/lab4_ILA/vivado，工程名称可取：TestILA ，添加文件夹 src ~~子文件夹中~~的 4 个设计文件和 1 个约束文件。

3．标记探测信号

在代码中，对需用 ILA 探测的信号，在其定义前加前缀(*MARK_DEBUG="TRUE"*) ，不管是 reg 型还是 wire 型，接口信号或者内部变量，都可以添加。

标记后的顶层文件（FlowingWaterLight.v）如下。

```
module FlowingWaterLight(clk,reset,direction,led);
   parameter sim=0;
   input clk,reset,direction;
   output[7:0] led;
// ------------------   分频计数器实例----------------//
    wire pulse2Hz;
    (*MARK_DEBUG="TRUE"*) wire[25:0] div_q;
    counter_n #(.n(sim?10:5000_0000),.counter_bits(sim?4:26))
         div_inst(.clk(clk), .en(1), .r(0), .q( div_q),.co(pulse2Hz));
  //------------------    可逆计数器实例   ------------------//
     (*MARK_DEBUG="TRUE"*) wire[2:0] q;
     counter_up_dn #(.counter_bits(3))   counter_up_dn_inst(
       .clk(clk),.en(pulse2Hz),.u_d(direction),.r(reset),.q(q));
  // ------------------     3 线-8 线器实例   ------------------//
    decode   decode_inst(.din(q),.out(led));
endmodule
```

4．创建 ILA 内核

（1）在 Vivado 工程界面中，如图 4.66 所示，在 Flow Navigator 窗口下，点击 PROJECT MANAGER⇨IP Catalog 选项。然后在在 Text Editor 窗口的 IP Catalog 选项卡中，找到并依次展开 Debug & Verification、Debug，并且双击 ILA（Integrated Logic Analyzer）选项，将会弹出如图 4.67 所示的 ILA 内核生成向导。

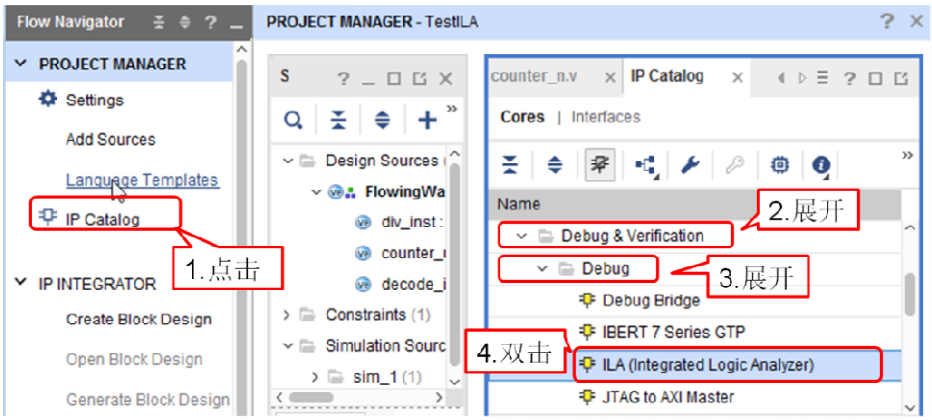


图 4.66　ILA

（2）ILA 参数配置

接下来就要配置 ILA 内核，点击图 4.67 的左上角“Documentation”就可以链接到 IP 核的帮助文档 data_sheet，如果对 IP 核的使用不是很懂可以点击此处获取 IP 核的详细信息。

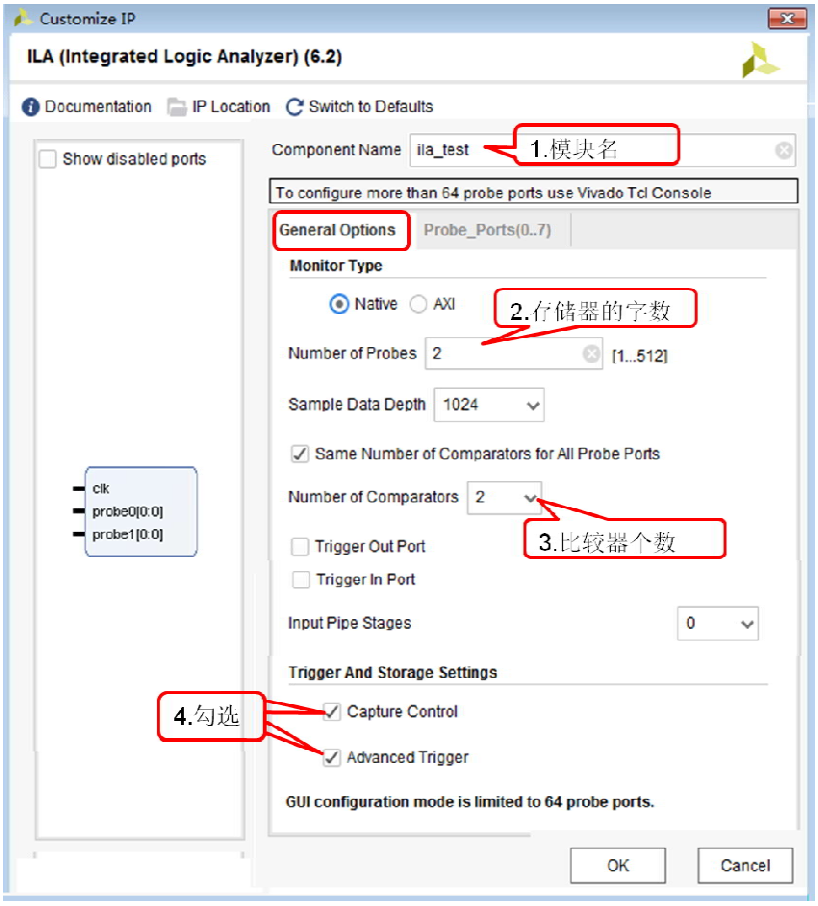


图 4.67　ILA 的基本参数配置

下面详细介绍参数配置。

如图 4.67 所示，在 General options 选项中配置 ILA 内核基本参数。

①图中最上面的“Component Name”可以给 IP 核取一个模块名：ila_test。注意，一旦生成了 IP 核，就算打开 IP 核进行修改，也不能再更改这个 IP 核的模块名了，所以取名需谨慎。

②“Monitor Type”是输入信号的格式，如果是 AXI 总线，就可以选择“AXI”，如果不是就选“Native”。

③“Number of Probes”是要探测的信号的通道数，本例共有两组信号需探测，所以就在方框里输入 2 即可。

④“Sample Data Depth”是采集深度，深度越大意味着能看到的信息量越多。但是采集的数据都是要存储在芯片内的 RAM 里，所以深度选择得越大占用的资源就越多，用户要根据自己芯片的情况选择采集深度。本例选采集深度为 1024。

⑤勾选“Number of Comparators for All Probe Ports”后，会自动选择比较器个数。

⑥“Trigger Out Port”和“Trigger In Port”是用于触发，可以不选，在后面调试时可以灵活使用。

⑦“Trigger And Storage Setting”是数据捕获的设置，一般直接勾上即可。

ILA 内核基本参数配置完毕后，点击 Probe_Ports 选项卡，在 Probe_Ports 这个页面，配置输入端口，如图 4.68 所示。因在“Number of Probes”中用设置了 2 个探测通道，这里就会出现了 2 个通道。因为需探测的信号 bit_q 和 q 分别为 26 位和 3 位信号，所以 PROBE0、PROBE1 通道的数据宽度分别填写 26 和 3 即可。

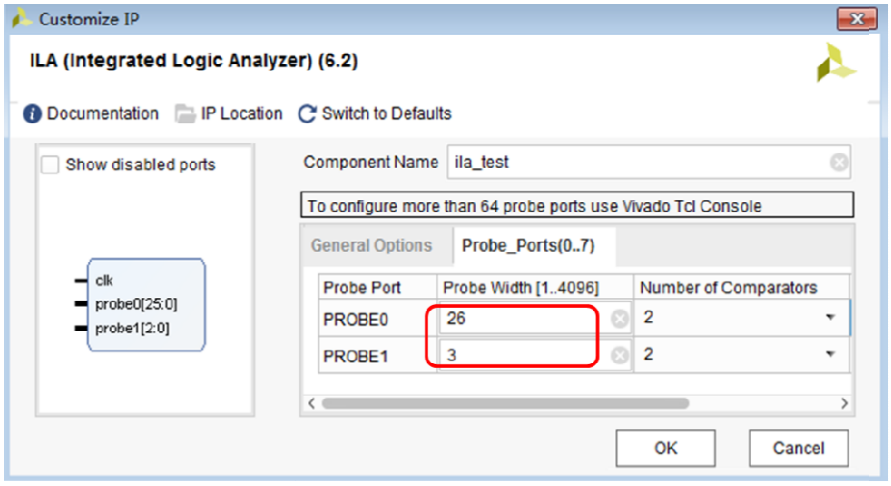


图 4.68　ILA 的探测端口配置

（3）ILA 参数配置完毕后，点击图 4.68 界面的 OK 按钮，在可能弹出的界面中点击 OK 按钮进入 ILA 内核生成的最后一个对话框，在此对话框中，选择综合方式 Out of context per IP 后，单击底部的 Generate 按钮，即可完成 ILA 内核生成创建。

5. 嵌入 ILA 内核

将刚才生成的 ILA 内核嵌入工程设计中，打开顶层文件（FlowingWaterLight.v），在 endmodule 前加入下面的 ILA 内核实例调用。

```
ila_test    ila_inst(
  .clk(clk),
  .probe0( div_q),
  .probe1(q));
```

6．编程下载

对工程进行综合、实现、生成 FPGA 编程比特流，然后下载到 FPGA 开发板。下载完成后，Vivado 界面发生变化，将会出现一个逻辑分析仪界面。

7. ILA 内核的触发条件和采集模式配置

①执行菜单 Window⇨Debug Probes 命令，界面会多出一个 Debug Probes 窗口，如图 4.69 所示。将用于触发条件的信号“拖入”右侧的 Trigger Setup 窗口，本例 div_q 和 q 两组信号都用于触发条件，故将 div_q 和 q 两组信号都拖入到 Trigger Setup 窗口。

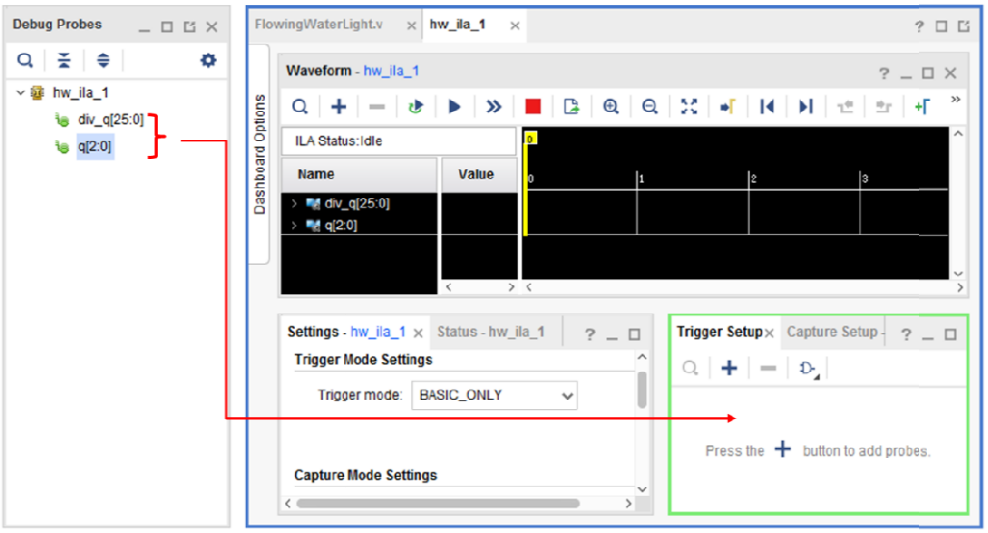


图 4.69　ILA 的触发信号的加入

② 如图 4.70 所示，在 ILA 设置窗口里，点击 Settings 选项卡，并选择触发模式：BASIC ONLY。然后，在 Trigger Setup 窗口设置触发条件：div_q= =49999999 和 q= =5。

本例触发条件设置目的是分析 5×10^7 分频器进位输出前后的状态转换情况。注意：设置触发条件时，应注意编码方式（Radix）的选择。因为这里 49999999 和 5 为为无符号数，所以 Radix 应选择为[U]（unsigned）。

③ 在 Capture Model Settings 栏中选择采集模式（BASIC）、采集深度（1024）和填写触发位置（3）。

采集深度表示需要采集多少数据；而触发位置即满足触发条件的采集数据在波形显示的坐标位置，也可理解为在波形显示时，显示保留满足触发条件前的采集数据个数。

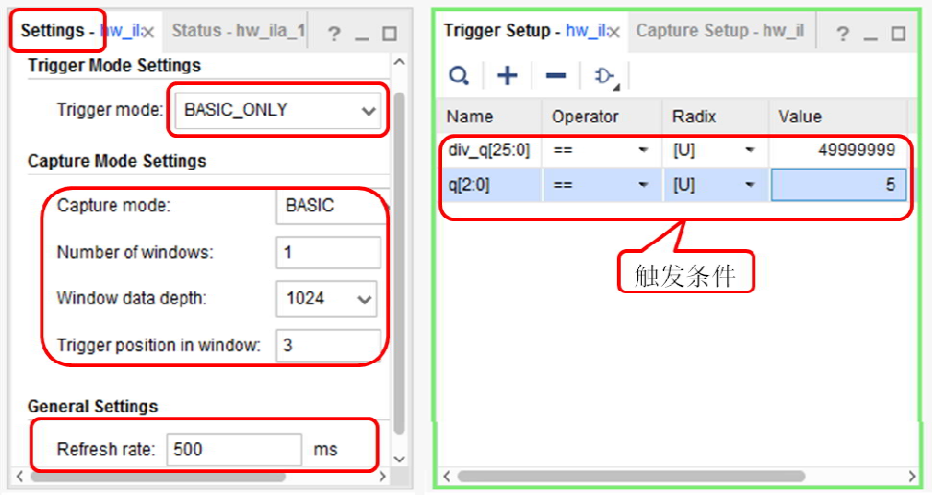


图 4.70　ILA 内核配置界面

8.采集数据

单击工具栏的开始采集运行图标“▶”，开始采集数据，采集结束后，可在 Waveform 窗口中观察采集到的信号波形，图 4.71 所示为信号波形的开始的一小部分（一共有 1024 个数据）。

注意：波形中信号的编码方式应为无符号数。

读者可更改采集深度、触发位置和触发条件等参数，重新采集数据，以加深对这些参数的理解。

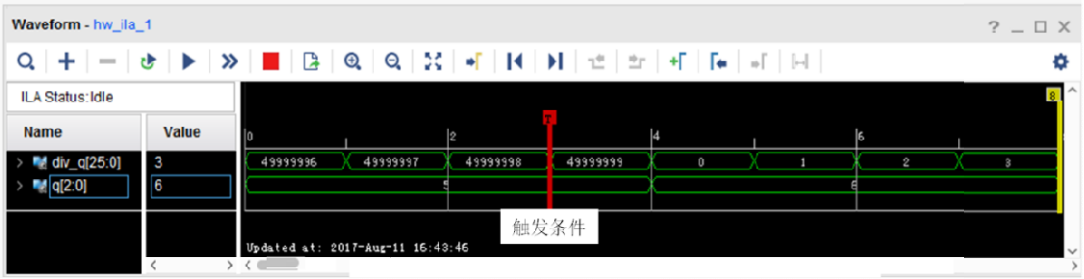


图 4.71　逻辑分析结果

至此，ILA 的逻辑分析步骤全部完成。

五、思考题

（1）什么是 ILA 内核？使用 ILA 的目的是什么？

（2）ILA 内核的数据、时钟和触发信号的作用分别是什么？